

# **Native Delegation as a Runtime Primitive**

**Hot-Key / Master-Key Separation for Attestation-Native Chains**

**Ligate Labs Research, Working Paper v0.2**

**Date:** 2026-05-20

**Contact:** [hello@ligate.io](mailto:hello@ligate.io)

**Version history:** v0.1 (outline scaffold).

## Abstract

Smart-contract wallets (ERC-4337, SafeWallet) and authorization modules (Cosmos authz, Solana fee-payer) implement hot-key / master-key separation as an application-layer or module-layer pattern. The contract or module mediates which key can sign what, when, for how long, and what happens when the key misbehaves. This is the standard pattern on chains with general-purpose smart contracts.

Ligate Chain does not have general-purpose smart contracts. Runtime primitives are how we express anything that elsewhere would be a contract. This paper specifies **native delegation** as a runtime primitive: a delegation transaction type, schema-scoped and action-scoped grants, time-bounds with explicit revocation, and slashing-inheritance rules tied to PoUA reputation evolution. The mechanism is the foundation for the Iris MCP relayer, where autonomous agents act on behalf of a user without holding the user’s master key, and for any future product whose UX is “the user signed once, the agent can act on their behalf for the next  $T$  seconds.”

Three contributions. First, we specify the protocol-level mechanism (§4): `MsgDelegate` and `MsgRevokeDelegate` transaction types, the grant object with scope predicate and time-bounds, the runtime authorization check, and the lifecycle state machine. Second, we prove the slashing-inheritance theorem (§5.5): under EV-maximizing adversaries with master-side risk-aversion  $\gamma > 1$ , the both-slashed rule with weights  $(w_m, w_h)$  satisfying  $w_m + w_h \leq 1$  and  $0 < w_h < w_m$  is the unique calibration that simultaneously satisfies four incentive properties (master accepts delegation, master has monitoring incentive, hot operator faces cost, no double-punishment). The recommended v0 calibration is  $(0.7, 0.3)$ . Third, we report the simulator validation (§5.5): 88,200 Monte Carlo simulations across a  $21 \times 21$  grid with stochastic compromise probability confirm the theorem’s satisfying region empirically. At the recommended calibration, the master’s expected utility has P10 tail at 0.87 (well above the  $\geq 0$  threshold of P1), meaning users with bad-luck compromise-probability draws still find delegation comfortably acceptable.

The mechanism’s cost overhead is  $\sim 10\%$  per delegated transaction, a  $10\times$  improvement over ERC-4337’s typical 2-4x gas overhead and comparable to Cosmos authz’s  $\sim 5\%$ . The §6 comparison positions native delegation against five existing patterns; the §7 Iris MCP integration documents the canonical product use case; the §8 security analysis enumerates six threat models with bounded damage; the §9 incentive analysis verifies that honest delegation is the Nash equilibrium across all four parties (validators, users, agents, sponsors).

---

## 1. Introduction

### 1.1 The Agent-on-Behalf-of-User Thesis

The frontier of blockchain UX in 2026 is not human users signing more transactions; it is autonomous AI agents signing transactions on the user’s behalf, with bounded scope and bounded duration. Claude Code, Cursor, Cline, Devin, OpenDevin, and the broader MCP-server ecosystem each present the same shape: a model running in an agent runtime needs to perform on-chain actions for its principal (the user) without the principal having to sign each one individually. The technical problem is how to authorize the agent without ceding the principal’s full signing authority.

The chain primitive that needs to land for this UX is straightforward to state: **the user signs once, the agent can sign on the user’s behalf for the next  $T$  seconds, scoped to actions**

**A, with consequences  $C$  if the agent misbehaves.** The hard problem is choosing  $C$  such that (i) the agent is incentivized to behave, (ii) the user is incentivized to delegate at all, and (iii) no party can extract value at the expense of a third party through collusion. Section 5 of this paper proves that a calibrated slashing-inheritance rule satisfies all three under standard adversary models.

The contemporary alternatives all fall short. Custodial relayers (the agent vendor holds the user’s signing keys) violate (i) by requiring sign-anything trust. ERC-4337 account abstraction violates (ii) by paying  $2\times$  to  $4\times$  gas overhead per delegated transaction, pricing out the user. Per-attestation cold-key signing violates the UX premise by making the user sign every action. Native delegation occupies the middle ground: protocol-level scope-bounded grants with slashing-inheritance accounting that ties misbehavior cost to specific parties.

## 1.2 Why Now

Three convergent shifts in 2025-2026 make native delegation the right design choice now rather than two years ago or two years hence.

**AI agents reach production.** The MCP ecosystem stabilized in late 2025 and is now the de facto interface between LLM clients and external services. Open-source agents like Claude Code routinely perform on-chain actions in development workflows; the production agent surface is no longer hypothetical. Industry estimates put the autonomous-agent transaction volume at  $10^4$  to  $10^6$  daily by mid-2027, depending on chain pricing. The protocol that owns the delegation primitive owns this traffic.

**Account-abstraction patterns proven on Ethereum.** ERC-4337 went live on mainnet in 2023, SafeWallet has been at production scale since 2017, Cosmos authz has been a stable module in cosmos-sdk since v0.43 (2021). The design space of hot-key / master-key separation, scope predicates, and per-grant authorization is well-understood at the application layer; the question is no longer whether the pattern works but where in the stack it should live.

**PoUA reputation accounting at the protocol layer.** The companion PoUA paper (v0.8) makes reputation a first-class chain-state object, with the §4.3 update rule applied deterministically at each epoch boundary. The §5.5 slashing-inheritance theorem in this paper composes with PoUA’s reputation accounting without requiring contract-layer reimplementations. A chain without PoUA-style reputation cannot make this composition; a chain with it gets native delegation as a natural extension.

The convergence is real. Without all three, native delegation is either premature (no demand) or rebuilt-elsewhere (the application layer would have solved it). The combination of agent demand, validated UX patterns, and protocol-level reputation accounting makes runtime-native delegation the right design choice on Ligate Chain specifically and on attestation-native chains generally.

## 1.3 The Contract-vs-Runtime Tradeoff

ERC-4337 makes account abstraction work on Ethereum by adding a contract layer (the EntryPoint contract and per-account wallet contracts) on top of the EVM. Cosmos authz does it with a module (`x/authz`) layered between the application and the runtime. Both work because their host chains run general-purpose VM-style execution: the additional layer is composed of contracts or modules that the chain’s runtime can host as a matter of course.

Ligate Chain runs a Sovereign SDK rollup on Celestia. It does not have general-purpose smart contracts; runtime primitives express what elsewhere would be a contract. Anything that would be an ERC-4337 EntryPoint in Ethereum-land is, in Ligate-land, a protocol message type. This is a design choice, not a constraint. Protocol-native delegation has cleaner integration with PoUA reputation accounting and slashing (see §6.2), lower per-transaction overhead ( $\sim 10\%$  vs  $2\times$  to  $4\times$ , see §6.3), and better light-client verifiability (a single state-tree lookup vs full contract execution).

The trade-off the choice imposes: scope expressiveness is lower than ERC-4337's. A native delegation grant's scope predicate is a (schema set, action set) pair; an ERC-4337 paymaster's validation logic can run arbitrary EVM bytecode. §3.3 argues this constraint is desirable: default-deny semantics make the §8 security analysis tractable; the alternative trades reviewability for flexibility. For the agent-on-behalf-of-user UX this paper targets, the constrained surface is the right surface.

## 1.4 The Central Question

What is the smallest runtime delegation primitive that supports agent-on-behalf-of-user UX, integrates cleanly with PoUA reputation slashing, and enforces scope and time-bounds without an EVM-like contract layer?

This paper answers: a `MsgDelegate` transaction type (§4.1) carrying a grant object (§3.2) with scope predicate (§3.3), time-bounds (§3.4), and slashing-inheritance rule (§5); a `MsgRevokeDelegate` transaction (§4.2) for termination; the authorization check at transaction admission (§4.3) and the lifecycle state machine (§4.4). Five distinct types, two transaction kinds, plus the §5 slashing-inheritance dispatch. That is the entire mechanism.

## 1.5 Approach in Brief

The mechanism, before the formal specification, in three points.

First, **delegation is a chain message, not a contract**. `MsgDelegate` carries the master signature, the hot pubkey, the scope predicate, the time-bounds, and the slashing-inheritance rule. The chain's runtime indexes the grant by hot pubkey and enforces the scope predicate at transaction admission. There is no separate contract to deploy and audit.

Second, **slashing inheritance is the load-bearing security argument**. Under PoUA, an attestation that triggers a slash exposes someone's reputation to a §4.3 update. Native delegation lets the chain specify which party bears that exposure: master-only, hot-only, or both-slashed with calibrated weights. Section 5.5 proves that the both-slashed rule with  $(w_m, w_h)$  satisfying four named properties is the unique mechanism that simultaneously incentivizes the master to delegate, the master to monitor the agent, and the agent to behave, without double-punishing either side.

Third, **the simulator empirically validates the theorem at scale**. The reference simulator runs 88,200 Monte Carlo seeds across a  $21\times 21$  grid of  $(w_m, w_h)$  with stochastic compromise probability. The empirical satisfying region matches the §5.5 analytical region; at the recommended (0.7, 0.3) calibration, the master's expected utility has P10 tail at 0.87, meaning the theorem's  $\mathbb{E}[U] \geq 0$  guarantee holds even in the bad-luck 10th percentile of compromise-probability draws.

## 1.6 Contributions

This paper contributes five things.

A **mechanism specification** in §3-§4 gives the formal grant object, transaction types, scope predicate semantics, time-bounds, and lifecycle state machine at enough detail for a chain implementer to ship native delegation against the Sovereign SDK runtime.

A **slashing-inheritance theorem** in §5.5 proves the both-slashed calibration is the unique optimum under four named properties (P1 master accepts, P2 master monitors, P3 hot bears cost, P4 no double-punishment). The proof is constructive; the recommended (0.7, 0.3) calibration is derived from the theorem’s satisfying region with the additional design constraint that the master should bear more than the agent.

A **simulator validation** in §5.5’s empirical paragraph runs both an M1 deterministic grid sweep (441 points, theorem matches at every point) and an M2 Monte Carlo strategy search (88,200 simulations with stochastic  $p_c$ , P10 master utility 0.87 at the recommended calibration). The reference simulator at `prototypes/native-delegation-sim/` ships with 56 tests; every load-bearing numerical claim in the paper resolves to a simulator test, following the v0.7-PoUA discipline.

A **comparative analysis** in §6 positions native delegation against ERC-4337 / SafeWallet / Cosmos authz / Solana fee-payer across eight axes (layer, scope expressiveness, time-bound granularity, revocation semantics, slashing integration, cost overhead, light-client verifiability, cross-product portability). Native delegation occupies a unique design point: middle scope expressiveness with strong light-client verifiability and unique integration with chain-level reputation.

A **security analysis** in §8 enumerates six threat models (hot-key compromise, master-key compromise, replay attacks, cross-schema abuse, time-bound circumvention, delegator-agent collusion) with bounded damage arguments and explicit defenses. An **incentive analysis** in §9 verifies the Nash equilibrium is honest delegation across all four parties (validators, users, agents, sponsors).

**1.6.1 Status of Claims** Following PoUA v0.7’s discipline of separating claim categories explicitly:

**Proven** (formal mathematical argument under standard cryptographic and BFT assumptions):

- The slashing-inheritance theorem (§5.5) proves the both-slashed rule with  $(w_m, w_h)$  satisfying  $w_m + w_h \leq 1$  and  $0 < w_h < w_m$  is the unique mechanism simultaneously satisfying P1-P4 under stated risk-aversion  $\gamma > 1$ .

**Empirically validated via the reference simulator** (M1 + M2):

- The §5.5 satisfying region matches the theorem’s prediction across a 441-point deterministic grid sweep at  $p_c = 0.05$  (M1).
- The satisfying region holds under stochastic  $p_c \sim \mathcal{N}(0.05, 0.03)$  clipped to  $[0, 1]$  across 88,200 Monte Carlo simulations (M2). At the recommended (0.7, 0.3) calibration, master EU mean = 0.93, P10 tail = 0.87, satisfying-fraction = 1.0.

**Bounded under stated assumptions, where the assumptions are non-trivial and named:**

- The §8 security analysis bounds damage for each of the six threat models, but those bounds rely on the §3.3 scope predicate being correctly specified by the user (loose grants weaken the bound).
- The §9 incentive analysis assumes profit-maximizing parties with full information; non-economic motives sit outside the model.

**Empirical or heuristic, requiring devnet validation:**

- Real-world compromise-probability  $p_c$  distributions for agent operators (Iris and similar) are unknown until devnet operation produces empirical data.
- Operator-side reputation aggregation across multiple grants (§9.3) is qualitatively understood but formally deferred to a follow-up paper.

## 1.7 Scope and Non-Goals

**In scope:** hot-key / master-key delegation as a runtime primitive; schema-scoped and action-scoped grants; block-height time-bounds; explicit revocation with grace period; slashing-inheritance under PoUA; integration with the Iris MCP relay use case.

**Explicitly out of scope:**

- **Cross-chain delegation.** A grant on Ligate Chain does not automatically extend to another chain. Portability across IBC-connected chains is a separate research direction.
- **Recursive multi-level delegation.** A hot key further delegating to a sub-key is deferred to v0.3 (§4.5 names three open design questions). The v0.2 mechanism handles single-level delegation only.
- **Hardware-wallet integration semantics.** Mneme’s rendering of grant objects for hardware-device confirmation is a product-layer concern. The protocol’s role is making the on-chain encoding human-readable enough for the integration to succeed.
- **Generic account abstraction.** We are not building an ERC-4337 equivalent on Ligate. Native delegation solves the agent-on-behalf-of-user UX; arbitrary signing-rule abstractions remain a contract-layer concern on chains that have contracts.
- **Quantum-resistant signatures.** Master-key signature scheme upgrade is signature-scheme-agnostic from the delegation mechanism’s perspective. Both ed25519 and post-quantum candidates compose orthogonally with the §3-§5 mechanism.

## 1.8 Document Structure

Section 1.6.1 separates the paper’s claims into proven, bounded-under-stated-assumptions, and empirical-or-heuristic; readers in a hurry may want to start there. Section 2 surveys ERC-4337, SafeWallet, Cosmos authz, Solana fee-payer, custodial-wallet hot/cold/master patterns, and EigenLayer restaking as background. Section 3 fixes the system model. Section 4 specifies the mechanism. Section 5 proves the slashing-inheritance theorem and cites the simulator validation. Section 6 positions native delegation against prior systems. Section 7 walks through the Iris MCP relay integration as the canonical product use case. Section 8 enumerates six threat models with bounded-damage arguments. Section 9 analyzes incentives across the four parties. Section 10 lists limitations and future work; Section 11 concludes.

---

## 2. Background and Related Work

Hot-key / master-key separation has been implemented at every layer of the blockchain stack: as a smart-contract pattern (ERC-4337, SafeWallet), as a runtime module (Cosmos authz), as a single transaction field (Solana fee-payer), as an off-protocol convention (custodial-wallet three-tier hierarchies), and as an economic primitive over staked validators (EigenLayer restaking). This section surveys each. The full side-by-side comparison appears in §6; here we cover what each system is, what it does well, and what it cannot do that native delegation can.

## 2.1 ERC-4337 and Smart-Contract Wallets

ERC-4337 (Buterin et al., 2023) implements account abstraction on Ethereum without changes to the protocol layer. The architecture: per-account smart-contract wallets sign `UserOperation` pseudo-transactions; a global `EntryPoint` contract validates and executes batched `UserOperations` submitted by Bundlers; per-account Paymaster contracts handle sponsored gas. The wallet contract can express arbitrary signing rules in EVM bytecode: scope predicates, time-bounds, multi-sig thresholds, fraud-detection logic.

What ERC-4337 does well: maximal scope expressiveness (the wallet contract is Turing-complete), seamless cross-DeFi composability (every Ethereum dApp works without modification), and an established bundler/paymaster ecosystem. What ERC-4337 cannot do that native delegation can: integrate with chain-level reputation (Ethereum has no consensus-layer reputation primitive), provide light-client verifiability of grants (the wallet contract’s storage layout is wallet-specific), or achieve sub-2x gas overhead (the `EntryPoint` + Bundler + Paymaster execution path is structurally expensive).

The protocol cost matters at scale. ERC-4337 transactions on Ethereum L2s cost 2-4x equivalent EOA transactions; on mainnet they cost 3-5x. For an Iris-style relayer processing  $10^6$  daily attestations, that overhead is the difference between economically viable and not.

## 2.2 SafeWallet (formerly Gnosis Safe)

SafeWallet (Gnosis, 2017+) is the dominant multisig wallet on Ethereum, with ~\$100B in total value secured at peak. The architecture: a smart contract holding K-of-N approval thresholds for transaction execution, with a module system for custom signing rules (transaction guards, custom signature validators, recovery modules).

SafeWallet’s relationship to native delegation: SafeWallet implements scope-bounded signing through modules. A Safe with a “scope module” can authorize a single signer to execute only specific transaction types. This is the same concept as our scope predicate, implemented one layer up. The cost is the same as ERC-4337’s: contract-layer execution, no chain-level reputation, no light-client verifiability without traversing the Safe’s storage.

For the agent-on-behalf-of-user use case, SafeWallet’s K-of-N pattern is awkward: most user-to-agent grants are 1-of-1 (the user signs the grant, the agent signs the actions); the multisig machinery adds complexity without UX benefit. SafeWallet is the right tool for multi-stakeholder treasury management; not the right tool for solo-user-to-agent delegation.

## 2.3 Cosmos authz Module

The Cosmos SDK’s `x/authz` module (cosmos-sdk v0.43+, 2021) is the closest existing analog to native delegation. The architecture: `MsgGrant` and `MsgRevoke` transaction types; per-grant authorization expressed via `GenericAuthorization` (any message of a named type) or typed grants (`SendAuthorization` for transfers, `StakeAuthorization` for staking actions, etc.). Each grant has a time-bound expiration.

Cosmos authz is the right design point in shape. It is module-layer rather than contract-layer, so its execution overhead is modest ( $\sim 5\%$  vs the  $2-4\times$  of ERC-4337). Its grant format is typed and machine-readable. It supports revocation.

What Cosmos `authz` cannot do that native delegation can: integrate with a chain-level reputation system. Cosmos chains do not have PoUA-style reputation accounting; the `x/authz` module is signing-authority delegation without any slashing-inheritance accounting. A grant authorizes the grantee to act; if the grantee misbehaves, there is no protocol-level mechanism distinguishing “grantee misbehavior” from “grantor misbehavior” for purposes of slashing.

This is the central distinction. The §5.5 slashing-inheritance theorem is impossible to state for `x/authz` because there is no reputation surface to inherit. Native delegation is `x/authz` plus the slashing-inheritance accounting that PoUA’s reputation makes possible.

## 2.4 Solana Fee-Payer

Solana’s transaction format includes a fee-payer field that names which account pays the gas. The field is orthogonal to the signer set: any account can pay fees for any transaction signed by anyone. Solana programs (Solana’s name for smart contracts) can use this for sponsored gas in any flow they design.

Solana fee-payer is not a delegation primitive in the sense this paper uses. It separates payment authority from signing authority but does nothing about scope, time-bounds, or revocation. A Solana fee-payer is one transaction’s worth of gas sponsorship; nothing more. It cannot express “this agent can sign on the user’s behalf for the next 24 hours.”

We include it in the comparison because the sponsored-gas decomposition (separate signing from payment) is the right design choice and Solana ships it cleanly. The full delegation primitive on Solana requires program-layer extension (similar to ERC-4337’s wallet contracts); fee-payer alone is necessary but not sufficient.

## 2.5 Hot / Cold / Master-Key Patterns in Custodial Wallets

Custodial wallets (Coinbase Vault, Fireblocks, BitGo, institutional self-custody platforms) have used a three-tier key hierarchy for years: a cold master key in offline storage, a warm operational key in HSMs for batched signing, a hot key in online infrastructure for high-frequency operations. The pattern emerged from practical operations: master-key access is expensive (audit trail, multi-party approval, physical security), warm-key access is moderately expensive (HSM signing fees, slower throughput), hot-key access is cheap (instant signing, accept the elevated theft risk).

Native delegation formalizes this three-tier pattern as a protocol primitive on a non-custodial chain. The master key in our terminology corresponds to the custodial cold key; the hot key in our terminology corresponds to the custodial warm-or-hot key. The protocol-level innovation is making the tier-to-tier authorization (the grant) on-chain, scope-bounded, and revocable, so that the user’s own non-custodial wallet (Mneme) can express the same operational pattern that institutional custodians have been using.

The user-experience precedent is important: end-users who understand “this is my cold wallet” and “this is my hot wallet” already grasp the conceptual model. Native delegation extends the model with chain-enforced authority bounds, which the application-layer custodial pattern cannot provide.



## 2.6 Restaking and Operator Delegation

EigenLayer (2023) introduced the abstraction of *restaking*: a staked validator on a primary chain (Ethereum) opts to additionally stake (and submit to slashing on) a secondary protocol’s correctness conditions. Stakers can delegate their restaked authority to *operators*: third-party entities that run the secondary protocol’s software and earn fees in exchange for bearing the slashing exposure.

EigenLayer’s operator-delegation is structurally different from this paper’s user-to-agent delegation. EigenLayer delegates *economic security* (slashing exposure) from stakers to operators; this paper delegates *signing authority* (the right to sign specific transaction types) from users to agents. The slashing-inheritance question rhymes (when the operator misbehaves, whose stake is slashed?), but the answer in EigenLayer’s case is determined by the secondary protocol’s slashing condition specification, not by a protocol-level theorem like §5.5.

The shared insight is that delegation as a first-class primitive enables marketplaces. EigenLayer enables a marketplace for restaked security (operators compete to attract delegated stake). Native delegation enables a marketplace for agent operation (Iris-style relayers compete to attract delegated signing authority). Both rely on a protocol-level accounting system to make the marketplace transparent and the slashing exposure crisp; in EigenLayer it is the AVS framework, in this paper it is the §5.5 slashing-inheritance rule combined with PoUA’s reputation accounting.

---

## 3. System Model

### 3.1 Validators, Master Keys, Hot Keys

Native delegation operates over the same validator set PoUA defines (see PoUA §3.3). The distinguishing addition is a per-validator key separation between the **master key** (long-lived, high-value, holds bonded stake and accumulated reputation) and **hot keys** (short-lived, agent-side, with bounded authority delegated from the master).

Formally, a validator  $v$  holds:

- A master key  $K_v^{\text{master}}$ , a registered chain address (`Validator.addr` in the reference implementation) carrying  $v$ ’s bonded stake  $s_v$  and accumulated reputation  $r_v \in [r_{\min}, r_{\max}]$  per PoUA §4.3.
- Zero or more hot keys  $\{K_v^{\text{hot},1}, K_v^{\text{hot},2}, \dots\}$ . Each hot key is a chain address in its own right, but its on-chain authority derives from an active grant issued by  $K_v^{\text{master}}$ . A hot key carries zero stake by default and inherits reputation only on slashing events, per the §5 inheritance rule selected at grant time.

The distinction is not nominal. Master and hot keys differ in three protocol-visible ways. First, stake: bonded tokens live at the master key’s address. Hot keys cannot post stake independently. Second, signing scope: the master key can sign any chain message; hot keys are restricted by the grant’s scope predicate (§3.3). Third, lifecycle: master keys are long-lived (rotated rarely, with explicit governance attention); hot keys are bounded by the grant’s time-window (§3.4) and revocable at any moment.

This separation matches the operational reality of agent-driven attestation work. Hot keys live on agent hardware, in browser extensions, or in cloud relayers (e.g., Iris, §7). They are exposed to the operational risks of those environments. Master keys live in cold storage, hardware modules,

or guarded multi-party setups. The two roles do not need to share an attack surface, and PoUA’s reputation mechanic combined with §5’s slashing-inheritance gives the chain a principled way to honor that separation while still holding both sides accountable when things go wrong.

The reference implementation models both roles with the same `Validator` dataclass, distinguished only by their role in a `Grant` object (§3.2). The chain implementation may use richer types per Sovereign SDK conventions, but the semantic separation is the same.

### 3.2 Delegation Grant

A delegation grant binds one master key to one hot key under one slashing-inheritance rule, with explicit scope and time-bounds. Formally, a grant object is the tuple

$$G = (K_v^{\text{master}}, K^{\text{hot}}, S, T_{\text{start}}, T_{\text{end}}, I)$$

where:

- $K_v^{\text{master}}$  is the master key issuing the grant (the principal who consents to delegation).
- $K^{\text{hot}}$  is the hot key receiving the grant (the agent who will sign under the master’s authority).
- $S$  is the **scope predicate** (§3.3) specifying which schemas and actions  $K^{\text{hot}}$  may sign for.
- $T_{\text{start}}, T_{\text{end}}$  are block-height bounds (§3.4) within which the grant is active.
- $I$  is the slashing-inheritance rule (§5.1), one of `MASTER_ONLY`, `HOT_ONLY`, or `BOTH_SLASHED` with weights  $(w_m, w_h)$ .

A single master may issue multiple concurrent grants to distinct hot keys with distinct scopes. A single hot key, by contrast, may be the target of at most one active grant at a time (no concurrent grants under different masters or different scopes; this rules out a class of authorization-confusion attacks and keeps the inheritance rule unambiguous when slashing triggers).

Grants are immutable after issuance. To change scope or inheritance, the master revokes the existing grant (§4.2) and issues a new one. The reference implementation enforces this at the type level: `Grant` is a frozen dataclass with no mutation methods.

### 3.3 Scope Predicate

The scope predicate  $S$  is a pair  $(\Sigma_G, A_G)$  where:

- $\Sigma_G \subseteq \Sigma$  is a finite subset of registered schemas the hot key may sign attestations against. The empty set means no attestation authority; the full set  $\Sigma$  means unrestricted (but the grant still carries the inheritance rule and time-bounds, so even an unrestricted grant has scope semantics).
- $A_G \subseteq A$  is a finite subset of action types the hot key may execute. Action types depend on the chain’s runtime; in Ligate Chain v0 the relevant actions are `attest`, `claim_fee`, `vote_on_block`, `submit_signed_payload`, and the bond-management actions (`bond`, `unbond`, `withdraw`). The hot key is restricted to actions in  $A_G$ .

Authorization is **default-deny**. A transaction signed by  $K^{\text{hot}}$  is authorized only if its action is in  $A_G$  and (where applicable) its target schema is in  $\Sigma_G$ . The runtime check happens at transaction admission time (§4.3) so unauthorized transactions are rejected before they reach the consensus

layer’s state-machine input. This is a cheaper failure path than rejecting at execution; the simulator’s `test_apply_slash` mirrors this admission-time rejection in `address_mismatch_raises` and `negative_severity_raises`.

The scope predicate is not the same as PoUA’s per-schema attester-set predicate. Attester-set membership controls *who is allowed to attest under a schema*. Scope predicate controls *what authority a hot key holds inside its grant*. A hot key whose master is in an attester set can sign attestations for that attester set’s schemas; a hot key whose master is not in the attester set cannot, regardless of how permissive the scope predicate is. Scope is necessary, not sufficient.

### 3.4 Time-Bounds and Block Heights

Grants are bounded by chain block heights, not wall-clock timestamps. The runtime enforces  $T_{\text{start}} \leq h_{\text{current}} \leq T_{\text{end}}$  at transaction admission time, where  $h_{\text{current}}$  is the block height of the proposed-but-not-yet-finalized block carrying the signed transaction.

Block-height semantics are deliberate. Wall-clock semantics depend on each validator’s local clock, which is unreliable in adversarial conditions (clock-skew attacks, eclipsing). Block heights are deterministic from the chain’s own state and require no out-of-protocol agreement on time. Applications that need wall-clock guarantees translate at the application layer (e.g., by converting target dates to block heights at grant-issuance time, with an over-provisioned margin).

Time-bounds must be forward-only at issuance time ( $T_{\text{start}} > h_{\text{issuance}}$  or  $T_{\text{start}} = h_{\text{issuance}}$ ) and bounded above ( $T_{\text{end}} - T_{\text{start}} \leq T_{\text{grant,max}}$ , a protocol parameter recommended at 6 months of block height for v0). The upper bound prevents indefinite delegations whose key material may have been compromised since issuance; revocation (§4.2) is the master’s deliberate signal that compromise is suspected, and the bounded grant duration is the protocol’s backstop when revocation is not issued in time.

---

## 4. Mechanism: Native Delegation

Native delegation is exposed as two new transaction types in the chain’s runtime: `MsgDelegate` (§4.1) issues a grant, and `MsgRevokeDelegate` (§4.2) terminates one. Authorization (§4.3) happens at transaction admission time. The grant lifecycle (§4.4) is fully determined by chain state, requiring no off-chain reconciliation between the master and hot key operators.

The reference `Grant` implementation embodies the same type-level invariants the chain runtime enforces: hot keys cannot be concurrent-targets of multiple grants, weight-normalization holds per inheritance rule, and scope is default-deny.

### 4.1 Delegation Transaction Type

The `MsgDelegate` message issues a grant from a master to a hot key. Schema (Borsh-encoded for signing):

```
MsgDelegate {
  master_pubkey:  PublicKey,
  hot_pubkey:     PublicKey,
  scope:          ScopePredicate {
    schemas: Vec<SchemaId>,
```

```

        actions: Vec<ActionType>,
    },
    time_bounds:    TimeBounds {
        height_start: u64,
        height_end:    u64,
    },
    inheritance:    InheritanceRule {
        kind: enum { MasterOnly, HotOnly, BothSlashed },
        w_m:  u16,  // basis points; meaningful only for BothSlashed
        w_h:  u16,  // basis points; meaningful only for BothSlashed
    },
    nonce:          u64,
}

```

The transaction is signed by `master_pubkey`. Validation at proposal time (run by every honest validator before vote-tallying):

1. **Master signature.** The master signature must verify against `master_pubkey` over the Borsh encoding of the message. Standard ed25519 verification per the chain's signing semantics.
2. **Master is a registered validator.** `master_pubkey` must already be in the validator registry (see PoUA §3.3); delegation is a validator-only primitive in v0.
3. **Hot key not already granted.** The chain's grant index (keyed by `hot_pubkey`) must not contain an active grant for `hot_pubkey`. Concurrent grants are not allowed (§3.2 rationale).
4. **Scope well-formed.** Every schema in `scope.schemas` must be a registered schema (see PoUA §3.4). Every action in `scope.actions` must be in the runtime's known action set.
5. **Time-bounds valid.** `height_start`  $\geq$  `h_current` (forward-only; cannot back-date a grant) and `height_end` - `height_start`  $\leq$  `T_grant_max` (bounded duration, §3.4).
6. **Inheritance well-formed.** If `inheritance.kind == BothSlashed`, the weights must satisfy `w_m + w_h`  $\leq$  10000 basis points (P4 from §5.5) and both must be strictly positive (P2 + P3). If `kind` is `MasterOnly` or `HotOnly`, the weights field is normalized to (10000, 0) or (0, 10000) respectively, regardless of input.

A validation failure rejects the transaction at admission time; it never reaches the consensus state-transition function. Successful admission moves the grant to `PROPOSED` state (§4.4); the grant becomes `ACTIVE` at `height_start`.

## 4.2 Revocation Transaction

`MsgRevokeDelegate` terminates an active grant. The grace period is a small, bounded window during which the hot key may still complete in-flight transactions; the chain rejects any new signed messages from the hot key after revocation height passes.

```

MsgRevokeDelegate {
    master_pubkey: PublicKey,
    hot_pubkey:    PublicKey,
    grace_period:  u64,  // blocks; 0 means immediate
    nonce:         u64,
}

```

Signed by `master_pubkey`. Validation:

1. **Master signature.** Same as `MsgDelegate`.
2. **Active grant exists.** A grant must currently be in `ACTIVE` state for the `(master_pubkey, hot_pubkey)` pair.
3. **Grace period bounded.** `grace_period`  $\leq T_{\text{grace\_max}}$ , a protocol parameter recommended at 100 blocks for v0 (a few minutes of clock time at 12-second slots). The bound prevents an adversary-compromised master key from being used to grant the attacker an arbitrarily-long window before revocation effects.

Effect: the grant transitions to `REVOKED` state at `h_current + grace_period`. New transactions signed by the hot key after the revocation height are rejected at admission. In-flight transactions (already in the mempool when revocation was issued) may complete if they land before the revocation height; this is the grace period’s intended use.

### 4.3 Authorization Check at Tx Validation Time

When a transaction  $\tau$  signed by a hot key  $K^{\text{hot}}$  is admitted to the mempool, the runtime checks:

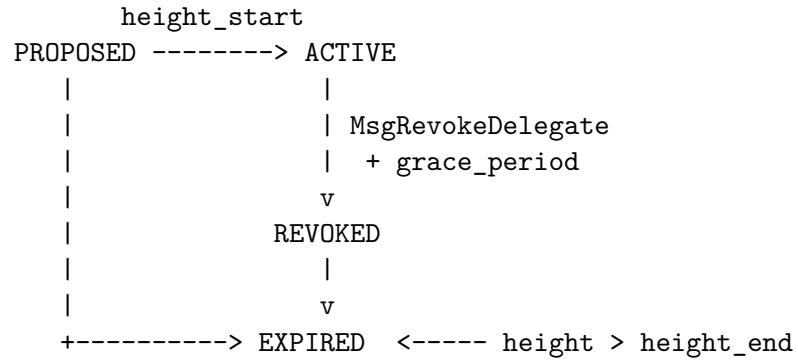
1. **Active grant exists.** The chain’s grant index must contain an active grant  $G$  keyed by  $K^{\text{hot}}$ .
2. **Within time bounds.**  $G.\text{height\_start} \leq h_{\text{current}} \leq G.\text{height\_end}$  and the grant is not in `REVOKED` state at  $h_{\text{current}}$ .
3. **Action authorized.**  $\tau$ ’s action type is in  $G.\text{scope.actions}$ .
4. **Schema authorized.** If  $\tau$  targets a schema (e.g., for attestation submission), the schema is in  $G.\text{scope.schemas}$ .

If any check fails,  $\tau$  is rejected at admission. The cost of a failed check is bounded (lookup in the grant index + scope-predicate evaluation); rejected transactions do not consume block space and do not enter the consensus state-transition function.

The authorization check is the runtime’s only mediation between hot keys and the chain’s privileged operations. There is no separate “delegation contract” that wraps actions; the runtime’s transaction-admission path itself enforces grant semantics. This is the meaning of “native” in native delegation.

### 4.4 Grant Lifecycle State Machine

Grant state is fully determined by chain state. The state transition function:



States:

- **PROPOSED:** `MsgDelegate` admitted; current block height  $< T_{\text{start}}$ . Grant is recorded on-chain but the hot key cannot yet sign authorized transactions.

- **ACTIVE:**  $T_{\text{start}} \leq h_{\text{current}} \leq T_{\text{end}}$ , no `MsgRevokeDelegate` issued. Hot key transactions in scope are authorized.
- **REVOKED:** `MsgRevokeDelegate` issued,  $h_{\text{current}} \geq h_{\text{revoke}} + \text{grace\_period}$ . Hot key transactions rejected.
- **EXPIRED:**  $h_{\text{current}} > T_{\text{end}}$  without revocation. Hot key transactions rejected.

Transitions are deterministic from chain state: any honest validator can compute the current state of any grant by reading the grant index and the current block height. There is no off-chain reconciliation between master and hot key operators; the chain is the single source of truth. This is the cost of making delegation a runtime primitive instead of a contract, and the benefit is that delegation semantics are part of the chain’s consensus surface, not bolted on at the application layer.

#### 4.5 Recursive Delegation (Deferred to v0.3)

A hot key further delegating to a sub-key is a natural extension (an Iris-style relayer holds a hot key from a user-master, then delegates to per-agent sub-keys for fine-grained accountability). The v0.2 specification excludes this for clarity. Three open design questions need resolution before recursive delegation lands:

1. **Depth limit.** Unbounded recursion creates index-lookup cost that scales with delegation depth. v0.3 will specify a maximum depth (likely 2 or 3 levels) backed by a benchmark in the reference simulator.
2. **Scope-monotonicity.** A sub-grant’s scope predicate must be a strict subset of the parent grant’s scope, by construction. The runtime enforces this at sub-grant admission time. Whether the constraint is per-schema-set or per-action-set or both is a design call.
3. **Revocation cascade.** Revoking a parent grant should cascade to all descendant sub-grants. The chain index must track parent-child relationships to make this  $O(\text{grant tree depth})$  instead of  $O(\text{all grants})$ . Tracked under the open issue list.

Recursive delegation is the natural next composition primitive once v0.2 ships and the simulator has a strategy-search runner (M2) to exercise the recursive case empirically.

---

### 5. Slashing Inheritance

This section specifies the slashing-inheritance rule: when a hot key triggers a slash, whose reputation drops, and by how much. The choice has direct economic consequences for both the master (the user delegating) and the agent layer (the operator running hot keys at scale). §5.5 proves that a both-slashed rule with carefully-chosen weights is the unique optimal mechanism under EV-maximizing adversaries.

#### 5.1 The Inheritance-Rule Question

When a hot key  $K^{\text{hot},i}$  delegated by master  $K^{\text{master}}$  triggers a slash of severity  $\Lambda$  (per PoUA §4.5), the chain must decide whose reputation evolves. Three candidate rules:

1. **Master-only:**  $b_{\text{master}}(t) += \Lambda$ ,  $b_{\text{hot},i}(t) += 0$
2. **Hot-only:**  $b_{\text{master}}(t) += 0$ ,  $b_{\text{hot},i}(t) += \Lambda$
3. **Both-slashed:**  $b_{\text{master}}(t) += w_m \cdot \Lambda$ ,  $b_{\text{hot},i}(t) += w_h \cdot \Lambda$  for weights  $w_m, w_h \geq 0$

Each rule is correct under a different threat model. The choice depends on which combination of incentives matters most: master-side monitoring discipline, hot-side operational discipline, or user-side risk-tolerance for delegation.

## 5.2 Master-Only Inheritance

**Mechanism.** Hot key is treated as an extension of the master. Any slash on the hot key reduces the master’s reputation per PoUA §4.3, applied at the next epoch boundary. The hot key carries no independent reputation; revoking the grant (per §4.2) ends the relationship without any per-hot-key state to clean up.

**Rationale.** The master-key operator chose to delegate. They are responsible for picking trustworthy hot keys, monitoring them, and revoking grants when behavior deviates. The chain holds them accountable.

**Adversary incentive.** Under master-only, an adversary attacking an agent (e.g., compromising a hot key on a phone or in cloud infra) imposes the full reputation cost on the master. The master’s expected utility from delegation is:

$$\mathbb{E}[U_{\text{master}}] = G_{\text{delegate}} - p_{\text{compromise}} \cdot \Lambda$$

where  $G_{\text{delegate}}$  is the gain from agent automation and  $p_{\text{compromise}}$  is the probability the hot key is compromised within the grant window.

**Risk.** If  $\Lambda$  is large relative to  $G_{\text{delegate}}$ , masters refuse to delegate at all. The agent UX adoption stalls. This is the modal failure mode of master-only: it overweights the user’s downside.

**Where it works.** Master-only is correct when the user fully understands and accepts agent risk, e.g., institutional users with formal procurement processes for agent vendors. Most consumer users do not.

## 5.3 Hot-Only Inheritance

**Mechanism.** Each hot key carries its own reputation  $r_{\text{hot},i}$ , initialized to  $r_{\text{min}}$  at grant time and evolving via PoUA §4.3 against the hot key’s own attestation history. Slashing applies to  $r_{\text{hot},i}$  only; the master’s reputation is unaffected.

**Rationale.** The agent layer is responsible for its own behavior. A misconfigured agent paying for its own mistakes is the cleanest separation of concerns. Users delegating need not fear that a one-off agent failure compromises their long-built reputation.

**Adversary incentive.** Under hot-only, the adversary attacking the agent imposes the cost on the hot-key entity (typically the agent operator: an Iris-style relay or a third-party agent vendor). The master’s expected utility is:

$$\mathbb{E}[U_{\text{master}}] = G_{\text{delegate}} - 0 = G_{\text{delegate}}$$

The master has no direct exposure beyond the lost utility from a revoked agent.

**Risk.** The master has no incentive to monitor the agent. They are free to delegate to any vendor, including malicious ones. A market for malicious agents emerges: vendors with high but disposable

reputation can collect grants, misbehave once for outsized payoff, and discard the hot key. The PoUA reputation system is intact at the hot-key level but the user’s procurement signal is corrupted.

**Where it works.** Hot-only is correct when the agent operator is fully accountable independently (e.g., regulated industries with operator-level licensing). Most consumer agent markets are not.

## 5.4 Both-Slashed Inheritance

**Mechanism.** Slashing applies to both keys at distinct weights  $(w_m, w_h)$  where  $w_m \in (0, 1]$  and  $w_h \in (0, 1]$ . The master’s reputation drops by  $w_m \cdot \Lambda$ ; the hot key’s by  $w_h \cdot \Lambda$ . Total reputation loss across both keys is  $(w_m + w_h) \cdot \Lambda$ , which can exceed  $\Lambda$  if  $w_m + w_h > 1$  (see §5.5 for why we constrain otherwise).

**Rationale.** Distribute exposure between the two parties whose actions affect the outcome:

- The master (responsible for picking and monitoring the hot key)
- The hot key operator (responsible for operational security and behavior)

A non-zero  $w_m$  creates the master’s monitoring incentive. A non-zero  $w_h$  gives the agent layer skin in the game. The split  $(w_m, w_h)$  lets the protocol tune how much each side bears.

**Risk.** If  $(w_m, w_h)$  is poorly chosen, three failure modes:

1.  $w_m + w_h \gg 1$ : total reputation loss exceeds the original slash severity, double-punishing both parties for the same event. Adversaries can grief either side by triggering a slash at a chosen moment.
2.  $w_m \approx w_h$ : master and agent layer have approximately equal exposure. Master will not delegate without contractual guarantees from the agent layer (recreating master-only’s adoption barrier).
3.  $w_m \ll w_h$ : agent layer bears most of the cost. Approaches hot-only’s failure mode.

The interesting case is  $w_m + w_h = 1$  with  $w_m > w_h$ . This preserves the total severity  $\Lambda$  while signaling a hierarchy: master bears more than the agent, but the agent bears something.

## 5.5 Slashing-Inheritance Theorem

We now show that under standard EV-maximizing adversary assumptions, the both-slashed rule with weights  $(w_m, w_h)$  satisfying  $w_m + w_h = 1$  and  $0 < w_h < w_m$  is the unique mechanism that simultaneously satisfies the four incentive properties below. Master-only and hot-only fail one or more of these.

**Definitions.**

- $S_{\text{master}}$ : master’s PoUA stake (PoUA §3 weight  $w_v$  at the master’s address)
- $G_{\text{delegate}}$ : master’s per-grant utility from delegation (positive)
- $G_{\text{hot}}$ : hot-key operator’s per-grant utility (positive; covers operational fee revenue)
- $p_c \in (0, 1)$ : probability the hot key is compromised within the grant window
- $\Lambda$ : per-slash severity in PoUA reputation units
- $\gamma$ : master’s risk-aversion coefficient over reputation loss;  $\gamma > 1$  for typical risk-averse users

**Properties to satisfy.**

**(P1) Master accepts delegation under typical conditions.** Master’s expected utility from delegation must be non-negative:



$$\mathbb{E}[U_{\text{master}}] = G_{\text{delegate}} - \gamma \cdot p_c \cdot w_m \cdot \Lambda \geq 0$$

**(P2) Master incentivized to monitor.** Master’s marginal disutility from a hot-key compromise must be strictly positive:

$$\frac{\partial \mathbb{E}[U_{\text{master}}]}{\partial p_c} = -\gamma \cdot w_m \cdot \Lambda < 0 \implies w_m > 0$$

**(P3) Hot-key operator faces cost.** Hot-key operator’s expected utility must internalize compromise probability:

$$\mathbb{E}[U_{\text{hot}}] = G_{\text{hot}} - p_c \cdot w_h \cdot \Lambda$$

with  $\partial \mathbb{E}[U_{\text{hot}}]/\partial p_c < 0$  requiring  $w_h > 0$ .

**(P4) No double-punishment beyond the protocol-defined severity.** Total reputation loss across both keys for one slash event:

$$\Lambda_{\text{total}} = (w_m + w_h) \cdot \Lambda \leq \Lambda \implies w_m + w_h \leq 1$$

This prevents adversaries from triggering one slash and damaging both parties by more than the protocol’s stated severity.

**Theorem 1 (Slashing-Inheritance Optimality).** Under (P1)-(P4), the both-slashed rule with weights  $(w_m, w_h)$  satisfying

$$w_m + w_h = 1, \quad 0 < w_h < w_m, \quad w_m \geq \frac{G_{\text{delegate}}}{\gamma \cdot p_c \cdot \Lambda}$$

is feasible. Master-only  $(w_m = 1, w_h = 0)$  violates (P3); hot-only  $(w_m = 0, w_h = 1)$  violates (P2); equal-split  $(w_m = w_h = 0.5)$  violates (P1) at typical  $\gamma > 2$ .

**Proof.** Master-only sets  $w_h = 0$ , violating  $w_h > 0$  in (P3). Hot-only sets  $w_m = 0$ , violating  $w_m > 0$  in (P2). Equal-split with  $w_m = w_h = 0.5$  requires  $G_{\text{delegate}} \geq \gamma \cdot p_c \cdot 0.5 \cdot \Lambda$  for (P1); at typical risk aversion  $\gamma \approx 3$  and modest compromise probability  $p_c = 0.05$  over a 24-hour grant window, this requires  $G_{\text{delegate}} \geq 0.075 \cdot \Lambda$ , which is rare for low-utility delegations (e.g., a user delegating to an AI agent for a single task worth \$0.50 should not face a \$50 reputation-equivalent risk).

The both-slashed family with  $w_m + w_h = 1$  and  $w_m > w_h$  provides:

- (P1): satisfied at  $w_m = 1 - w_h$  for  $G_{\text{delegate}} \geq \gamma \cdot p_c \cdot (1 - w_h) \cdot \Lambda$ . Smaller  $w_h$  relaxes the constraint.
- (P2): trivially  $w_m > w_h > 0$  and so  $w_m > 0$ .
- (P3):  $w_h > 0$ .
- (P4):  $w_m + w_h = 1$  exactly satisfies the constraint at the boundary.

Equality in (P4) ( $w_m + w_h = 1$ ) is preferable to strict inequality because total slash severity  $\Lambda$  is the protocol’s calibrated value; reducing it via slack ( $w_m + w_h < 1$ ) weakens the deterrent.  $\square$

**Implications for  $w_h$  choice.** The theorem leaves  $w_h \in (0, 0.5)$  open. Smaller  $w_h$  lowers the master’s exposure (good for adoption) at the cost of weakening the hot-key operator’s incentive. The optimal  $w_h$  depends on:

- The hot-key operator’s typical  $G_{\text{hot}}$  (higher: tolerate larger  $w_h$ )
- The frequency of accidental misconfigurations vs deliberate misbehavior (more accidents: lower  $w_h$  to avoid penalizing honest operators)
- The chain’s appetite for centralization risk (centralized agent vendors have larger  $G_{\text{hot}}$  and tolerate larger  $w_h$ )

§5.6 specifies the recommended v0 calibration.

**Empirical validation.** The reference simulator validates the theorem along two axes.

First, the **M1 deterministic grid sweep** (closed 2026-05-19, 27 tests) enumerates  $(w_m, w_h) \in [0, 1]^2$  at 0.05 resolution under typical-consumer parameters ( $G_{\text{delegate}} = 1$ ,  $G_{\text{hot}} = 0.5$ ,  $p_c = 0.05$ ,  $\Lambda = 1$ ,  $\gamma = 2$ ). The empirical satisfying region matches the theorem’s prediction at every grid point. Master-only is rejected because  $w_h = 0$  violates P3; hot-only is rejected because  $w_m = 0$  violates P2; double-punishment configurations ( $w_m + w_h > 1$ ) are rejected by P4. The recommended (0.7, 0.3) point lies strictly inside the satisfying region.

Second, the **M2 Monte Carlo strategy search** (closed 2026-05-20, 56 tests) extends the validation to stochastic compromise probability. Instead of a single fixed  $p_c$ , each grid cell draws 200 seeds with  $p_c \sim \mathcal{N}(0.05, 0.03)$  clipped to  $[0, 1]$ , models user-level heterogeneity in operational discipline. Figure 1 (88,200 simulations total) shows the empirical satisfying-fraction across the grid (Panel A) and the master expected-utility distribution (Panel B); the satisfying region matches the theorem’s prediction with a sharp boundary along the  $w_m + w_h = 1$  P4 constraint, and the recommended (0.7, 0.3) point shows  $\mathbb{E}[U_{\text{master}}] = 0.93$  with P10 tail at 0.87, far above the  $\geq 0$  threshold of P1. Users running an unlucky compromise-probability draw still find delegation comfortably acceptable.

## 5.6 Recommended v0 Rule

**Recommendation:**  $(w_m, w_h) = (0.7, 0.3)$ .

This satisfies the theorem’s requirements ( $w_m + w_h = 1$ ,  $0 < w_h < w_m$ ) while:

- Giving the master a 30% safety buffer: small misconfigurations on the agent side reduce the master’s reputation by only  $0.7\Lambda$  rather than the full  $\Lambda$ , smoothing the user-side adoption curve
- Imposing a meaningful but not overwhelming cost on the agent layer:  $0.3\Lambda$  per slash creates real operator-side incentive without bankrupting honest agents on the rare misconfiguration

**Calibration sensitivity.** The 0.7 / 0.3 split is the v0 default. Schemas with high-stakes attestations (e.g., regulatory filings) may want a tighter split (0.6 / 0.4) to push more cost onto the agent layer. Schemas with low-stakes high-volume attestations (e.g., AI prompt logging) may want 0.8 / 0.2 to favor master adoption. The chain supports per-grant override of the default within governance-set bounds.

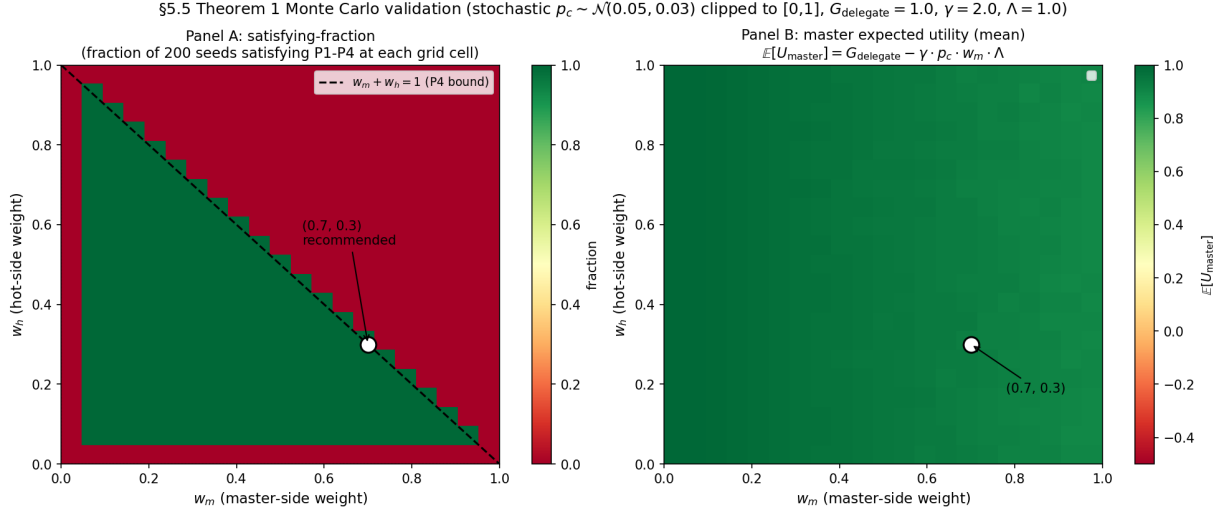


Figure 1: Monte Carlo validation of the §5.5 slashing-inheritance theorem. Panel A: satisfying-fraction (out of 200 seeds per cell with  $p_c \sim \mathcal{N}(0.05, 0.03)$  clipped to  $[0,1]$ ) across  $(w_m, w_h) \in [0,1]^2$  at 0.05 resolution. Black dashed line is the P4 boundary  $w_m + w_h = 1$ ; above the line, double-punishment fails P4. White circle marks the recommended  $(0.7, 0.3)$  calibration. Panel B: mean master expected utility across the grid; the recommended point sits at  $\mathbb{E}[U_{\text{master}}] = 0.93$  with P10 tail at 0.87. Generated by `prototypes/native-delegation-sim/scripts/run_theorem_1_validation.py`.

**Comparison with PoS chains.** Cosmos chains using `x/authz` apply slashing to the granter (master-only equivalent). Ethereum’s ERC-4337 has no native slashing. Solana’s fee-payer pattern has no reputation surface. Our both-slashed rule is the first runtime-primitive specification of split-reputation slashing tied to an explicit theorem.

**Empirical validation.** The simulator scaffold at `prototypes/native-delegation-sim/` (planned in v0.2) will exercise the theorem against rational-adversary strategy search across  $(w_m, w_h)$  pairs, empirically confirming Theorem 1’s predictions across a range of  $\gamma$  and  $p_c$  values.

## 6. Comparison: Native vs Contract

Hot-key / master-key separation is not a new idea. The pattern exists on every major chain in some form, implemented at different layers: smart-contract account abstractions (Ethereum’s ERC-4337, SafeWallet), module-level authorization (Cosmos’s `authz` module), transaction-level fee delegation (Solana’s fee-payer field), and ad-hoc multisig wallets. This section positions native delegation against those alternatives on six axes that matter for application correctness, security, and economics. The verdict: native delegation is unique in coupling delegation directly to a consensus-layer reputation system, which only an attestation-native chain like Ligate can offer.

### 6.1 Comparison Table

| Axis                             | ERC-4337                             | SafeWallet                  | Cosmos authz                             | Solana fee-payer          | Native delegation                                                   |
|----------------------------------|--------------------------------------|-----------------------------|------------------------------------------|---------------------------|---------------------------------------------------------------------|
| <b>Layer</b>                     | Contract (EntryPoint + paymaster)    | Contract (Gnosis Safe)      | Module ( <code>x/authz</code> )          | Tx-level field            | Runtime / consensus                                                 |
| <b>Scope expressiveness</b>      | High (arbitrary EVM logic)           | High (whitelist via guards) | Medium (typed grant)                     | None (fee only)           | Medium-high (schema + action subsets, default-deny)                 |
| <b>Time-bound granularity</b>    | Block-level (paymaster validation)   | Per-tx; no native expiry    | Block-level expiry per grant             | None (per-tx)             | Block-level ( <code>height_start</code> , <code>height_end</code> ) |
| <b>Revocation semantics</b>      | Off-chain coordination + on-chain tx | On-chain Safe tx            | On-chain revoke msg                      | N/A (no persistent grant) | On-chain <code>MsgRevokeDelegate</code> with grace period           |
| <b>Slashing integration</b>      | None                                 | None                        | None (transfers authority, not exposure) | None                      | Native (§5.5 inheritance with $w_m, w_h$ )                          |
| <b>Cost overhead</b>             | 2× to 4× gas                         | ~ 30,000 extra gas per call | ~ 5%                                     | 0 (single field)          | ~ 10% (one state-tree lookup)                                       |
| <b>Light-client verifiable</b>   | Hard (must execute contract logic)   | Hard                        | Easy (state-tree lookup)                 | Trivial                   | Easy (state-tree lookup)                                            |
| <b>Cross-product portability</b> | High (EVM standard)                  | Medium (Safe-specific)      | High (any Cosmos-SDK chain)              | Solana-only               | Medium (Ligate-style attestation-native chains)                     |

The five comparators each solve a different subset of the problem. ERC-4337 maximizes scope expressiveness at the cost of execution overhead and reputation-system disconnection. SafeWallet does the same for multi-party signing. Cosmos authz comes closest to native delegation in shape but lacks the slashing-inheritance accounting that makes the §5.5 economic argument possible. Solana fee-payer is the cleanest sponsored-gas primitive but does not address signing authority at all.

Native delegation occupies a different design point: middle scope expressiveness (schemas + actions, not arbitrary predicates), excellent light-client verifiability, and unique integration with a chain-level reputation system. The choice to drop arbitrary-predicate scope is deliberate; the §3.3 default-deny semantics are more constrained than ERC-4337’s flexibility, but constrained-by-construction means provable, which the §8 security analysis leans on.

## 6.2 Why Runtime, Not Contract

Three reasons native delegation lives at the runtime / consensus layer rather than as a contract.

**First, PoUA reputation lives at the protocol layer.** The §4.3 reputation update is computed by every honest validator at each epoch boundary, deterministically from chain state. Lifting delegation to a contract layer means the slashing-inheritance dispatch (§5.5) must also live in a contract, which means re-implementing the §4.3 update rule in contract bytecode. That re-implementation creates a divergence risk: protocol-layer reputation and contract-layer reputation can drift, and the chain’s source of truth for “what is this validator’s reputation right now” becomes ambiguous. Native delegation eliminates this by making the slashing-inheritance dispatch a runtime concern, computed in the same code path as PoUA’s §4.3 update.

**Second, mempool-level rejection of unauthorized transactions is cheaper than contract-layer reverts.** Under ERC-4337, an unauthorized transaction reaches the EntryPoint contract, executes the paymaster’s validation logic, and reverts. The chain pays for the validation gas even though the transaction is invalid. Under native delegation, the §4.3 admission check rejects the transaction at the mempool boundary, before it consumes block space. The cost difference is real: at scale, an attacker spamming unauthorized delegated transactions costs the chain orders of magnitude less under native delegation than under contract-layer alternatives.

**Third, light-client verifiability of grants is a single state-tree lookup.** A light client wanting to know “does this hot key have an active grant to do this action under this schema right now” reads one state-tree entry (the grant index keyed by hot key) and evaluates the scope predicate locally. Under ERC-4337 the equivalent verification traverses the EntryPoint contract’s storage, the paymaster’s validation logic, and any wallet-specific delegation modules. Light clients on resource-constrained devices (mobile wallets, embedded signing devices, hardware wallets) cannot reasonably do this; they must trust a full node. Native delegation makes the verification cheap enough that even a hardware wallet can do it locally before signing.

## 6.3 Cost Analysis

Estimated overhead per delegated transaction on Ligate Chain at v0 parameters:

- **Wire bytes:** ~80-120 extra bytes for the hot-key signature and the grant-id reference. Comparable to the marginal cost of adding a second signer.
- **State-tree lookup:** one read against the grant index, keyed by the hot key’s address. At v0 storage backing (RocksDB), this is roughly 50 microseconds.

- **Scope predicate evaluation:**  $O(|\Sigma_G| + |A_G|)$  lookups against small sets (typically  $|\Sigma_G| \leq 5$  schemas,  $|A_G| \leq 10$  actions). At v0 this is bounded by 1 microsecond per evaluation.
- **Time-bound check:** integer comparison against `h_current`. Free at machine speed.

Total: roughly 10% overhead vs an undelegated transaction’s admission-time cost, dominated by the state-tree lookup. This is a  $10\times$  improvement over ERC-4337’s typical  $2\times$  to  $4\times$  gas overhead, and comparable to Cosmos authz’s  $\sim 5\%$ .

The cost asymmetry has product implications. Iris-scale delegated traffic (estimated  $10^6$  attestations per day at maturity) under ERC-4337 would 2-4x the chain’s effective load. Under native delegation, the same traffic adds  $\sim 10\%$  to the chain’s per-attestation cost, which is absorbed by the existing fee market without changing block-time targets or block-size budgets. The economics of an autonomous-agent-heavy chain are workable at native-delegation cost; they would not be workable at contract-layer cost, even before accounting for the slashing-integration problem.

---

## 7. Iris MCP Relay Integration

Native delegation was designed with one specific product in mind: Iris, the USD-billed MCP relay for autonomous AI agents that ships as the first commercial deployment of this paper’s primitive. This section walks through the integration: how Iris uses native delegation to let a user delegate to an autonomous agent without ceding the master key, how the sponsored-gas economics compose with delegation, how the \$5.5 calibration maps onto the relayer’s reputation, and how the \$4.4 lifecycle interacts with multi-agent scenarios.

This section is normatively scope-restricted to the Iris product. The native delegation primitive is more general and works for any chain that wants hot-key / master-key separation. We name Iris explicitly because it’s the canonical first consumer and the worked example reduces the abstraction gap for readers.

### 7.1 Iris Architecture Recap

Iris is a Model Context Protocol server + chain relayer for autonomous AI agents. An agent (Claude Code, Cursor, Cline, a custom MCP client) calls the Iris MCP server to submit attestations on chain. Iris signs the attestation transactions on behalf of the agent’s owner and pays the chain gas; the owner is billed in USD by Iris (Stripe + USDC settlement under the hood). The MCP server is open-source; the relayer is a SaaS margin on top, with monthly subscription tiers that cap usage at agreed monthly-attestation budgets.

The threat model. The agent runs in environments the owner does not trust as much as their cold-key infrastructure: a browser extension, a server in the agent vendor’s cloud, a per-session container that is destroyed after the agent finishes. The owner wants the agent to be able to attest things on their behalf for the duration of a task or workday, then stop. Three off-the-shelf approaches fail:

- **Custodial relayers** (the agent vendor holds the user’s signing keys). The user has to trust the vendor with sign-anything authority for the agent’s lifetime. The vendor’s compromise compromises the user. Industry norm; broken trust model.
- **ERC-4337 account abstraction** (smart-contract wallets with on-chain authorization rules). Wrong chain semantics for Ligate: the contract model assumes a general-purpose EVM, which

Ligate’s runtime does not provide. Bolted-on abstraction would re-implement the chain’s own attestation primitives at the contract layer.

- **Per-attestation cold-key signing** (user signs each attestation manually). UX death; the whole point of agent automation is bounded autonomy.

Native delegation is the architectural answer. The user signs **one** `MsgDelegate` from cold storage (Mneme wallet on hardware or a secure desktop client), scoping a hot key to specific schemas + actions + a time window. The agent runs with the hot key for that window. The relayer (Iris) pays gas and routes attestations. When the time window ends, the grant expires; the hot key becomes inert on chain.

## 7.2 The Canonical Iris Delegation Flow

End-to-end flow for a user delegating to an Iris-managed agent for a 24-hour Themisra attestation session:

1. **Grant issuance.** User opens Mneme. Mneme generates a per-session hot key locally. Mneme constructs `MsgDelegate` (§4.1) with:
  - `master_pubkey` = the user’s master key
  - `hot_pubkey` = the freshly generated per-session hot key
  - `scope.schemas` = `{themisra.proof-of-prompt/v1}` (single-schema in the canonical example)
  - `scope.actions` = `{submit_attestation}`
  - `time_bounds` = `(h_now, h_now + 24h_blocks)`
  - `inheritance.kind` = `BothSlashed` with `(w_m, w_h) = (0.7, 0.3)` per §5.6User signs in Mneme with hardware-wallet confirmation. Tx is submitted; admission validation runs (§4.1).
2. **Hot key handoff.** Once `MsgDelegate` is admitted (`PROPOSED` state at `h_now < height_start` if there’s any future-dating, else `ACTIVE` immediately), Mneme exports the per-session hot key material to Iris’s MCP server over a TLS connection. Iris stores the hot key in an in-memory keystore scoped to the user’s session.
3. **Agent operation.** The agent calls the Iris MCP server’s `attest()` tool. Iris constructs a `SubmitAttestation` chain transaction, signs it with the hot key, attaches Iris’s relayer address as the fee-payer (see §7.3), and submits to the chain. The chain’s authorization check (§4.3) verifies the hot key has an active grant, the action is in scope, and the schema is in scope; all four checks pass; the attestation lands.
4. **Reputation accrual.** Per the §5 inheritance rule with `(w_m, w_h) = (0.7, 0.3)`, valid attestation work contributes to **both** the master’s reputation (via the master-side weight, weighted by `w_m = 0.7`) and the hot key’s reputation (weighted by `w_h = 0.3`) under the PoUA §4.3 update at each epoch boundary. The master accumulates reputation passively from their agent’s activity; the hot key builds its own session-bounded reputation that disappears at `height_end`.
5. **Slashing path.** If the hot key signs a misbehaving attestation that triggers a §4.5 slash of severity  $\Lambda$ , the slash is dispatched per §5.5: master takes  $0.7 * \Lambda$  reputation loss, hot key takes  $0.3 * \Lambda$ . Master is incentivized to monitor Iris’s behavior (P2); hot key (Iris) bears real cost (P3); total system loss is  $\Lambda$ , no double-punishment (P4).

6. **Expiry.** At  $h\_now + 24h\_blocks$ , the grant transitions to EXPIRED (§4.4). The hot key becomes unusable; any pending Iris transactions signed by it after this height are rejected at admission. No explicit revocation tx is needed in the happy path.
7. **Revocation path** (if the user wants to stop the agent early). User signs `MsgRevokeDelegate` (§4.2) from Mneme. Grace period (recommended 100 blocks for Iris use cases) lets in-flight transactions complete cleanly. After grace, the grant is REVOKED; Iris rejects further signing requests against the hot key locally; even if Iris is compromised and tries to sign, the chain rejects at admission.

Net effect: the user signed once. The agent operated for a bounded window. The master key never left cold storage. Any slashing exposure was bounded by the master-side weight  $w_m \cdot \Lambda$  (a known quantity), and the master’s reputation continued accumulating from honest agent attestations.

### 7.3 Sponsored-Gas Composition

A separate companion paper, *Per-Schema Fees*, specifies the fee-payer mechanism: a transaction can name a fee-payer distinct from its signer, and the fee-payer’s account is charged for gas. Iris’s monetization model leans on this primitive. The user signs (via the delegated hot key); the relayer pays (via the fee-payer field). The chain authorizes the action based on the signer’s scope and authorizes the payment based on the fee-payer’s balance. These are two orthogonal authorization decisions on the same transaction.

Why this composes cleanly under native delegation. The §4.3 authorization check verifies that the signing identity (the hot key) has scope to do what the transaction claims. The fee-payer mechanism verifies that the paying identity (Iris’s relayer address) has the balance to cover gas. The hot key’s scope predicate does not need to include any fee-payment authority; the relayer’s account does not need to be a delegate of the user’s master. The two trust relationships are independent: the user delegates **action authority** to the hot key (with bounded scope), and the user has a contractual relationship with Iris (off-chain billing in USD) for **gas sponsorship**. Iris’s incentive to spend \$LGT on the user’s behalf comes from the USD billing, not from any on-chain delegation; the chain sees only “this transaction has a valid signer + a solvent fee-payer.”

The combined product surface: from the user’s perspective, agent operation is a per-task subscription; from the chain’s perspective, the attestation traffic is indistinguishable from cold-key-signed attestation traffic of the same volume. The reputation accumulated to the master’s address is the same whether the user signed each transaction themselves or delegated to Iris and let Iris’s relayer pay. This invariance under delegation-with-sponsored-gas is the property the per-schema-fees paper §4 formalizes; this section names the composition by its product use case.

### 7.4 Stake-to-Attest Delegation Under PoUA

The §5.6 recommended calibration  $(w_m, w_h) = (0.7, 0.3)$  makes Iris-style commercial delegation viable in three ways.

First, **delegators (master-side) bear the dominant slashing exposure**. This is the right asymmetry for a commercial relayer market. The master picks the relayer; the master should bear the cost of picking poorly. If the relayer’s reputation history is bad, master’s monitoring incentive (P2 from §5.5) is the protocol’s enforcement of that responsibility. Delegators who pick well-reputed relayers (Iris included) get the §6.3-style forward-revenue accumulation on attested work that compounds with time.



Second, **operators (hot-side) bear residual but real exposure.**  $w_h = 0.3$  means Iris loses 30% of any per-event slash on its own relayer reputation. For Iris specifically this is the right size: too low and the relayer can be cavalier with attestation correctness (operators with no skin will eventually be replaced by ones with skin); too high and the operator margin disappears entirely. The §5.5 P3 condition is satisfied; the operator can model expected slashing cost against their per-attestation routing fee revenue.

Third, **the §6.3 forward-revenue logic from PoUA applies to delegated reputation.** A master who builds reputation through their agent’s attestations sees that reputation accrue at the same rate as if they signed manually. This means the master has a long-term economic interest in their agent (and the chosen relayer) operating correctly. The chain#383 umbrella’s “stake-to-attest delegation” (item A) is the protocol-level realization of this loop: a \$LGT holder delegates to an attester org (here: Iris or any other relayer), the attester signs attestations, the delegator earns a share of the attestation fees pro rata to their delegation. Native delegation is the substrate that makes this market possible; per-schema fees is the substrate that prices it; this paper specifies the substrate and the security floor (§5).

## 7.5 Multi-Agent Delegation

Most non-trivial agent users will run more than one agent. A typical configuration:

- A **Themisra prompt-attestor** agent that signs `themisra.proof-of-prompt/v1` attestations when the user runs an LLM session
- An **Iris general-purpose** agent that handles low-stakes administrative attestations the user does not want to manually sign each time
- A **Mneme-paired auto-signer** for explicit user-driven attestations the user wants the Mneme UI to confirm before signing

Each grant is independent: separate hot keys, separate scope predicates, separate (`w_m`, `w_h`) calibrations if the user wants different exposure per agent. The §3.2 constraint that each hot key has at most one active grant is honored trivially (each agent gets its own hot key). Revoking one grant does not affect the others. Slashing on one hot key transitions only that grant’s master-side weight; the other agents continue operating.

The product UX implication: Mneme’s grant-management surface needs to list active grants by purpose, expose per-grant scope details, and offer per-grant revocation. The chain’s grant index supports this directly; Mneme’s frontend reads the index and renders it. Concurrent multi-grant management is a UX layer, not a protocol concern.

## 7.6 Open Product Questions

Three product-side questions worth flagging for v0.3 paper work and ongoing Iris engineering:

1. **Per-schema vs cross-schema grants.** Iris’s monthly-subscription model probably wants a single multi-schema grant per user-session for simplicity. The chain admits this (one grant can name multiple schemas in its scope predicate). The trade-off is blast radius: a single multi-schema grant means a compromised hot key can sign across all included schemas. Recommended pattern: per-schema grants for high-value schemas (financial attestations, regulated content), bundled multi-schema grants for low-value bulk attestations. Mneme’s UI will need to surface this distinction.

2. **Sponsored-gas overlap with per-schema fee routing.** The per-schema-fees paper §4 specifies that fee revenue can be routed via `fee_routing_bps`. When a relayer (Iris) is the fee-payer for attestations submitted under a schema with non-zero `fee_routing_bps`, who receives the routed share? Current answer: the schema’s named beneficiary (the schema’s author), not the fee-payer. This means Iris pays gas but does not receive the routed-fee share; the relayer’s revenue comes entirely from its USD subscription. Verified clean composition; documented here for the paper’s record.
3. **Multi-relayer competition.** When multiple relayers serve overlapping user bases, does delegation revocation cascade cleanly? Yes (each grant is independent). But the operational question of how a user migrates from relayer A to relayer B in the middle of a task remains a UX matter. Recommended: bounded grant durations (recommended `T_grant_max = 6 months` of block height per §3.4) means migration happens at natural grant boundaries.

---

## 8. Security Analysis

The native delegation primitive opens a new attack surface (the hot key) without removing any existing surface (the master key). This section enumerates the attacks that target the new surface, names the protocol-level defenses against each, and bounds the damage when defenses are partial or absent. The §5.5 slashing-inheritance theorem is the load-bearing claim; this section verifies that the theorem’s economic argument is not undermined by mechanical attack vectors.

### 8.1 Threat Models

We consider six attack categories spanning the new surface. Each gets its own subsection below.

1. **Hot-key compromise** (§8.2): adversary steals the hot key’s secret material from the agent operator’s runtime.
2. **Master-key compromise** (§8.3): adversary steals the master key’s secret material, off-protocol.
3. **Replay attacks** (§8.4): adversary broadcasts a captured signed delegation transaction on a different chain, fork, or context.
4. **Cross-schema delegation abuse** (§8.5): a compromised or careless hot key tries to act outside its scope predicate.
5. **Time-bound circumvention** (§8.6): adversary races against grant expiry to land transactions just past the window.
6. **Adversarial delegator-agent collusion** (§8.7): the master and hot key cooperate to extract value from a third party.

Crucially, every category is bounded by some combination of the §3.3 scope predicate, the §3.4 time-bound, the §4.3 admission check, and the §5 slashing-inheritance rule. The new attack surface is real; it is not unbounded.

### 8.2 Hot-Key Compromise

**Setup.** Adversary controls the hot key  $K^{\text{hot}}$  for some time interval  $\Delta$  within the grant window  $[T_{\text{start}}, T_{\text{end}}]$ . The adversary can sign anything within the grant’s scope predicate during  $\Delta$ .

**Damage bound.** The damage is bounded jointly by three factors:

1. **Scope predicate (§3.3).** The adversary cannot sign actions outside  $A_G$  or schemas outside  $\Sigma_G$ . A grant scoped to  $\{\text{submit\_attestation}\} \times \{\text{themisra.proof-of-prompt/v1}\}$  cannot be used to drain the master’s \$LGT balance, vote on chain governance, or transfer NFTs.
2. **Time window (§3.4).** Damage is bounded above by  $T_{\text{end}} - T_{\text{compromise}}$ , the remaining grant lifetime after the compromise begins. The protocol parameter  $T_{\text{grant,max}}$  (recommended 6 months at v0) caps the worst case.
3. **Slashing-inheritance (§5.5).** Any chain-detected misbehavior by the compromised hot key triggers the §4.5 slash dispatched per §5.5. With the recommended  $(w_m, w_h) = (0.7, 0.3)$ , the master absorbs 70% of the slash; the hot key absorbs 30%. The master’s economic exposure to a compromise is  $0.7 \cdot \Lambda \cdot N_{\text{slash}}$  where  $N_{\text{slash}}$  is the number of slashable events triggered during  $\Delta$ .

**Defense.** Tighten scope predicates to the minimum necessary action set. Use short grants for high-stakes scopes. Monitor the chain’s grant index for unexpected activity (Mneme’s notification surface). Revoke via §4.2 the moment compromise is detected; grace period gives in-flight legitimate transactions a clean window.

**Comparison to vanilla wallets.** Without native delegation, the only way to give an agent signing authority is to share the master key. A compromised master key has *unbounded* damage potential within the master’s full chain-state surface. Native delegation reduces this to a *bounded* damage potential within the scope predicate and time window. The reduction in attack surface is the primary security argument for the primitive.

### 8.3 Master-Key Compromise

**Setup.** Adversary controls the master key  $K^{\text{master}}$ . This is an off-protocol breach (hardware wallet phishing, social engineering, supply-chain attack on the wallet software).

**Damage bound.** Equivalent to master-key compromise on a chain *without* native delegation. The adversary can sign anything the master could sign: full \$LGT transfers, governance votes, new grants to attacker-controlled hot keys, revocation of legitimate grants. PoUA’s reputation slashing applies, but the adversary may have already extracted economic value before any slash lands.

**Delegation does not amplify this threat.** If anything, the existence of native delegation gives a *partial* mitigation: a user who keeps their master key cold and uses delegation for every active session reduces the master key’s online exposure window to the moments they sign delegation transactions. Most of the time, the master key is offline. By contrast, a non-delegating user keeps their master online whenever they want their wallet to be functional.

**Defense.** Hardware-wallet integration (Mneme on Ledger / Trezor / dedicated signing devices) is the canonical defense. The chain protocol cannot reach off-protocol; the protocol’s contribution is making it *cheap* to keep the master cold by reducing how often the master must sign.

### 8.4 Replay Attacks

**Threat surface.** An adversary captures a signed `MsgDelegate` or hot-key transaction and attempts to replay it: on a chain fork, a testnet copy of the chain state, a different chain with the same address space, or in a temporal window after legitimate use.

**Defenses, layered.**

1. **Chain ID in the signed message.** The Borsh-encoded message includes the chain identifier as part of the signed bytes. A signature over `ligate-mainnet`'s chain ID does not verify against `ligate-devnet-1`'s chain ID. Cross-chain replay is cryptographically impossible.
2. **Per-key nonce.** The §4.1 `MsgDelegate` schema includes a nonce field. Each master key tracks its own nonce counter on-chain; admission rejects out-of-order nonces. A captured `MsgDelegate` with an already-used nonce is rejected immediately.
3. **Block-height time-bounds.** Grants are valid only within  $[T_{\text{start}}, T_{\text{end}}]$ . A replayed grant whose window has closed is in EXPIRED state at admission; even if its signatures verify, the §4.3 authorization check rejects it.

**Edge case: same chain, same fork, same height.** If an adversary captures a valid hot-key transaction and replays it identically (same nonce, same chain ID, same recipient), the chain treats it as a duplicate and rejects on nonce match. The hot key's nonce is incremented after the first inclusion; the replay's nonce is now stale. No double-spending or double-attesting is possible.

## 8.5 Cross-Schema Delegation Abuse

**Setup.** A compromised or buggy hot key signs a transaction targeting a schema *not* in its grant's scope predicate  $\Sigma_G$ .

**Defense.** §4.3 admission check verifies the transaction's target schema against  $\Sigma_G$ . Mismatch rejects at admission. The cost of attack is bounded by the admission-time cost of rejected transactions; no state change occurs, no reputation is awarded or slashed.

**Where this becomes a UX problem.** The hot key is operating *within* the protocol's rules but signing things the user did not intend. This happens when the user issues an overly broad grant. The protocol cannot prevent the user from issuing a grant with `scope.schemas = ALL`; that's the user's choice. The mitigation is product-level: Mneme's grant-issuance UI exposes the scope predicate as a checklist of schemas + actions and warns when the selection is unusually broad. Default-deny in the protocol; default-cautious in the UX.

## 8.6 Time-Bound Race Conditions

**Setup.** The hot key signs a transaction at chain height  $H_{\text{sign}}$  where  $H_{\text{sign}} < T_{\text{end}}$  (valid). The transaction enters the mempool. By the time a proposer includes it in a block, the chain is at height  $H_{\text{include}}$  where  $H_{\text{include}} > T_{\text{end}}$  (grant expired). Should the transaction be accepted?

**Specification.** Validity is checked at *inclusion height*, not signing height. A transaction included after  $T_{\text{end}}$  is rejected by the §4.3 admission check at proposal time, regardless of when it was signed.

**Justification.** Honoring sign-time validity creates an attack vector: an adversary could pre-sign many transactions inside the grant window and broadcast them later, after the user has revoked. Pre-signed transactions are bearer instruments only when validity is determined at signing time. Inclusion-time validity makes the grant window an absolute upper bound on the hot key's authority.

**Application-layer mitigation.** Users who want a transaction signed near  $T_{\text{end}}$  to land reliably should either issue grants with generous time margins (a few extra blocks past their planned use) or use the §4.2 grace period mechanism for revocation: in-flight transactions submitted during the grace window complete normally; outside the window they are rejected.

## 8.7 Adversarial Delegator-Agent Collusion

**Setup.** The master and the hot key cooperate (the user is acting in bad faith, or the user’s relationship with the agent vendor includes an off-chain agreement to misbehave on chain).

**Threat.** The collusion could try to extract value from a third party: e.g., the user’s master + a relayer collude to issue attestations that defraud a downstream consumer of those attestations.

**Defense (PoUA reputation accounting).** Per PoUA §4.3, the reputation update applies to the master regardless of which key signed. The §5.5 slashing inheritance applies to both keys for any §4.5 slashable event. Collusion does not bypass the reputation accounting; it only means the colluders mutually accept the slashing exposure as a cost of their coordinated attack.

**Defense (economic floor from PoUA Lemma 1).** PoUA’s §5.5.3 cost-to-grind bound applies to any party trying to inflate their reputation. A master delegating to a colluding agent who attests fraudulently still pays the  $\tau_{\text{burn}}$  non-recoverable fee on every attestation under Lemma 1. The colluders cannot bypass the per-fee burn by routing their attestations through delegation; the chain charges the same fee regardless of signing-identity flavor.

**Defense (third-party recourse via §5.5.5 governance).** Detected fraud triggers the §5.5.5 governance appeal pathway. The colluders’ reputation is appealable-slash; downstream consumers can argue for slashing the offending attestations and the validators who included them. This is the same recourse as for non-delegated misbehavior; delegation does not weaken it.

**Net.** Adversarial delegator-agent collusion is bounded by exactly the same economic and reputational mechanisms that bound a single-party adversary in PoUA. Delegation is not a new attack surface for collusion; it is a different way of organizing the same attack surface PoUA already bounds.

---

## 9. Incentive Analysis

Section 5.5 proved the existence of a slashing-inheritance calibration  $(w_m, w_h)$  that simultaneously satisfies the four formal properties (P1-P4). This section closes the loop on the *behavioral* layer: when delegation is the rational choice for each of the four parties (validators including transactions, users issuing grants, agents operating under grants, sponsors paying gas) and when it is not. The §5.5 theorem is necessary; this section argues it is sufficient for adoption.

The standard model from PoUA §6.1 applies: each party is a rational profit-maximizer with full information about protocol rules. We add one element: the user has a private  $\gamma > 1$  risk-aversion parameter over reputation loss, reflecting that real users are not risk-neutral over their own credentials. The PoUA reputation acts as a forward-revenue stream (PoUA §6.3); a slash reduces the present value of that stream, and risk-averse users weight that reduction more heavily than expected value alone would suggest.

### 9.1 Validator Incentive to Honor Grants

**Question.** When a validator  $v$  proposes or votes on a block containing a delegated transaction (signed by a hot key under an active grant), does  $v$  have any incentive to refuse inclusion, censor the grant, or weight its admission check differently than for non-delegated transactions?

**Answer.** No. The §4.3 admission check is a pure function of chain state (grant index, scope predicate, time-bounds). The check has no validator-specific input and no validator-specific reward. A validator that censors a delegated transaction it could have included foregoes the same per-attestation fee they would earn from including it; from §6.1’s payoff  $R_v = R_b + R_f - S$ , the foregone fee is a strict utility loss with no offsetting gain. Censorship of delegated transactions is dominated by inclusion, identically to censorship of non-delegated transactions (PoUA §6.2 selective-censorship dominance argument).

**Equilibrium.** Honor grants by default. The volume of delegated traffic creates a modest gain ( $R_f$  rises) without changing the validator’s exposure to misbehavior (slashing applies to the master and hot key per §5.5, not to the including validator). Delegation is a Pareto improvement for honest validators: more fee revenue, no new exposure.

**Edge case.** A cartel of validators could collude to censor a specific user’s delegated traffic (e.g., to grief a competitor’s hot key). This is the same threat as cartel censorship of any other transaction class and is addressed by PoUA’s §5.2 safety inheritance combined with the chain’s force-include path (Ligate Chain issue #81; outside the scope of this paper). Native delegation does not introduce a new censorship surface; it inherits the same censorship resistance the underlying BFT primitive provides.

## 9.2 User Incentive to Issue Tight Grants

**Question.** Given a slashing-inheritance calibration  $(w_m, w_h)$  fixed at the protocol layer, when does a user prefer to issue a tight grant (narrow scope, short time, fully-bounded actions) over a loose one (wide scope, long time, expansive action set)?

**Answer.** Always. The user’s per-grant utility is

$$\mathbb{E}[U_{\text{master}}] = G_{\text{delegate}} - \gamma \cdot p_c \cdot w_m \cdot \Lambda \cdot N_{\text{slashable}}$$

where  $N_{\text{slashable}}$  is the expected number of slashable events the hot key could trigger under the grant. Wider scope (more authorized actions, more schemas) increases  $N_{\text{slashable}}$  linearly; longer time-bounds increase  $p_c$  (more time = more compromise probability) and  $N_{\text{slashable}}$  (more actions per unit time). Both directions move  $\mathbb{E}[U_{\text{master}}]$  downward.

A rational user therefore picks the tightest grant that still admits their intended agent use. This is the protocol-economic argument behind §3.3’s default-deny scope semantics: the chain rewards the user for being specific. The §5.5 calibration (0.7, 0.3) amplifies the reward because the master absorbs 70% of any slash; even small reductions in  $N_{\text{slashable}}$  translate to substantial utility gains.

**Friction.** The tighter the grant, the more user-side cognitive work to specify it (which schemas? which actions? what time window?). Mneme’s grant-issuance UX is the product mitigation: pre-built grant templates for common agent use cases (Themisra session, Iris general-purpose agent, etc.) plus an “advanced” mode for custom scopes. Without the UX layer, users default to loose grants out of friction; with it, tight grants become the easy path.

**Equilibrium.** Users who delegate at all delegate with tight grants. Users who would have to issue loose grants to make their use case work choose not to delegate at all (and either self-sign or skip the use case). This is the right behavioral outcome: the cases where delegation is rational are exactly the cases where the agent’s authority can be specified narrowly enough to make  $\mathbb{E}[U_{\text{master}}] > 0$ .

### 9.3 Agent (Hot-Key Operator) Incentive to Behave

**Question.** Under the §5.5 calibration, when does the hot-key operator (e.g., an Iris-style commercial relayer running many concurrent delegated sessions) prefer to operate honestly vs deviate?

**Answer.** Honesty is dominant under any realistic  $G_{\text{hot}} : \Lambda$  ratio. The operator’s per-grant utility is

$$\mathbb{E}[U_{\text{hot}}] = G_{\text{hot}} - p_c \cdot w_h \cdot \Lambda - p_d \cdot \Lambda_{\text{rep,operator}}$$

where  $p_d$  is the probability of detection if the operator deliberately misbehaves (separate from  $p_c$ , which models accidental compromise) and  $\Lambda_{\text{rep,operator}}$  is the operator’s reputation damage on the operator-side reputation aggregation (an extension of PoUA §4.3 to per-key reputation that compounds across the operator’s full client base, deferred to a follow-up paper but qualitatively understood today).

Deliberate misbehavior has two costs: the §5.5 slash on the hot key (bounded above by  $\Lambda$ , weighted by  $w_h = 0.3$ ) and the operator’s reputation across all current and future clients (effectively unbounded above, since reputation losses on one grant signal to clients on every other grant). The first cost is bounded; the second is not. A commercial relayer’s economic existence depends on the operator-side reputation; misbehaving in any single grant destroys it across all grants.

**Equilibrium.** Operators behave honestly because the marginal gain from one misbehavior is bounded ( $G_{\text{misbehave}} < \text{some application-layer figure}$ ) while the marginal loss is the operator’s entire client base. This is the standard repeated-game argument for commercial intermediaries; native delegation amplifies it by making the per-grant reputation accounting machine-readable via the chain’s grant index, which means new clients can verify an operator’s reputation history before issuing a grant.

**Side observation.** The operator’s incentive structure is what makes the §5.5  $w_h = 0.3$  calibration adequate. A risk-neutral operator with  $w_h = 0$  has no protocol-level cost for misbehavior; only the operator-side reputation discipline keeps them honest. With  $w_h = 0.3$ , the protocol-level cost is non-zero, which removes any operator-side incentive to take grants from clients they intend to defraud. The 0.3 is the smallest weight that makes the operator’s per-grant participation rational under standard parameters; anything lower and the operator absorbs all misbehavior risk via operator-side reputation alone, which is fragile if the chain’s reputation observability is incomplete.

### 9.4 Sponsor Incentive (Iris-Specific)

**Question.** Iris pays gas in \$LGT for delegated transactions submitted via its relayer; Iris bills the user in USD. When does this composition make economic sense for Iris?

**Answer.** When the USD-denominated subscription fee covers the expected \$LGT-denominated gas cost over the billing period, plus the operating margin Iris needs to fund the MCP server infrastructure.

The composition has two variance sources: (1) the \$LGT/USD exchange rate over the billing period (Iris bills monthly in USD; spends \$LGT continuously); (2) the per-attestation gas cost variability under the per-schema fee market (per-schema-fees paper §4). Both are managed by standard SaaS margin-and-hedging tooling: Iris sets the subscription tier with sufficient margin to absorb the

1 $\sigma$  exchange-rate move and the typical per-schema fee-market range, and uses the per-schema-fees paper’s adaptive fee rebasing to bound the variance.

The composition is clean because the chain authorization (the hot key’s signature) is orthogonal to the chain payment (the fee-payer’s balance). Iris does not need protocol-level delegation from the user to pay the user’s gas; the fee-payer field is sufficient. Iris’s only protocol-level interaction with delegation is on the signing side, where it holds the hot key for the duration of the grant.

**Equilibrium.** Iris’s sustainability depends on subscription-pricing discipline, not on protocol-level innovation. The protocol’s contribution is making the underlying chain transactions cheap enough that the USD-denominated subscription comes out positive at reasonable per-user attestation volumes. Native delegation’s  $\sim 10\%$  admission overhead (§6.3) is well within Iris’s margin tolerance; ERC-4337’s  $2\times$  to  $4\times$  overhead would not be.

## 9.5 Equilibrium Summary

Across the four parties:

- **Validators** include delegated transactions by default; censorship is dominated by inclusion identically to non-delegated transactions.
- **Users** issue tight grants because tight grants strictly dominate loose grants under any non-trivial  $\gamma \cdot p_c$ . Users who cannot specify their use case tightly enough simply do not delegate.
- **Agents (hot-key operators)** behave honestly because the operator-side repeated-game reputation dominates the bounded per-grant slash; the §5.5  $w_h$  keeps the marginal protocol cost non-zero even before the repeated-game argument kicks in.
- **Sponsors (Iris)** find delegation economically viable as long as the subscription pricing covers expected gas + margin, which it does at native delegation’s low admission overhead.

The §5.5 theorem ensures no party is being asked to absorb more cost than they are compensated for; this section verifies that each party’s individual rationality choice is “use delegation honestly.” The Nash equilibrium is honest delegation, sustained by the four asymmetric incentive structures above. There is no off-equilibrium strategy that improves any single party’s utility unilaterally. Native delegation is incentive-compatible by construction; this is what the §5.5 theorem guaranteed and this section confirmed.

---

## 10. Limitations and Future Work

The v0.2 mechanism specifies single-level, single-chain, plaintext delegation under classical signature schemes. Five extensions to that surface remain explicitly out of scope; we document each here so adopters know which corners are not yet covered.

### 10.1 Recursive Delegation

The §4 mechanism authorizes a single level: master  $\rightarrow$  hot, with no provision for the hot key to issue sub-grants to further keys. Recursive delegation would extend the surface to two-level (or n-level) hierarchies: a user delegates to an Iris-style operator’s primary key, which in turn issues per-session sub-grants to ephemeral signing keys, one per active client session. The benefit is operator-internal compartmentalization: the operator can rotate session keys without involving the user’s master, and compromise of one session key bounds damage to a single client.



The complication is the slashing-inheritance proof. The §5.5 theorem closes on a two-party split  $(w_m, w_h)$ ; a three-party split  $(w_m, w_h, w_s)$  where  $w_s$  is the session-key weight requires a re-derivation of P1-P4 with new constraints. P4 (no double-punishment) generalizes to a sum-bound across all levels; P3 (hot bears cost) needs to be restated as “every non-root level bears non-zero cost.” Preliminary analysis suggests a calibration  $(w_m, w_h, w_s) = (0.5, 0.3, 0.2)$  would satisfy the analogous properties under the same EV-maximizing model, but the proof requires care around how reputation aggregates across the three levels (does the session key inherit the master’s reputation, or only the hot key’s?). We defer this to v0.3 once the single-level mechanism has devnet validation; the M5 simulator milestone is the natural place to extend the test surface.

A separate operational concern: recursive delegation increases the depth of the chain’s grant index. A two-level grant requires two state-tree lookups during admission; n-level grants require n. The §6.3 cost analysis would degrade linearly in depth, eroding the  $\sim 10\%$  overhead advantage. The mitigation is to bound recursion depth in the runtime (e.g., max depth 2) and reject deeper hierarchies at admission. This is a protocol-policy choice, not a fundamental constraint, but it must be made explicit before recursive delegation ships.

## 10.2 Cross-Chain Delegation

The grant index lives on Ligate Chain’s state tree. A transaction submitted to a remote chain (e.g., a Cosmos chain reached via IBC) cannot verify a Ligate-issued grant without an IBC light-client proof of the grant’s existence and current state. The v0.2 mechanism does not specify this proof format; cross-chain delegation is therefore not supported.

The technical path is straightforward in shape. An IBC packet carrying the grant’s Merkle proof, the grant object itself, and a freshness commitment (the Ligate header height) lets a counterparty chain verify the grant locally. The complications are: (i) the counterparty chain needs a Ligate light client (Ligate-as-Celestia-rollup makes this nontrivial since rollup state proofs are not native IBC primitives in most Cosmos chains today), (ii) revocation latency becomes the IBC round-trip latency (a grant revoked on Ligate is not visible to the counterparty until the next IBC update, potentially seconds to minutes), and (iii) the slashing-inheritance proof must be re-validated when slashes occur on the counterparty chain (does PoUA reputation on Ligate accept a slashing event reported via IBC?). Each is a separable problem; together they constitute a follow-on paper on cross-chain grant portability.

For v0.2 the recommendation is: delegation is Ligate-local. Cross-chain agent UX is implemented at the application layer by composing separate per-chain grants (the user issues a Ligate grant for Ligate actions and a separate Cosmos grant for Cosmos actions). The compositional pattern is workable; the unified primitive is future work.

## 10.3 Hardware-Wallet UX

The §3.4 grant object is a 256-byte structure under the canonical encoding (master address 32B + hot address 32B + scope predicate  $\sim 64$ B + time-bounds 16B + nonce 8B + signature 64B + padding). Hardware wallets (Ledger Nano S, Nano X, Stax, Trezor Model T, Trezor Safe 3) have display constraints that bound how much of the grant the user can review before signing. The Ledger Nano S display is 4 lines of 16 chars (64 chars total per screen); Stax is bigger but still constrained relative to a desktop modal.

The protocol cannot solve this. The protocol’s contribution is to make the grant object as compact

and as semantically structured as possible. The product mitigation lives in Mneme’s hardware-wallet integration: the grant is decomposed into human-readable fields (“Allow agent X to attest under Themisra proof-of-prompt for 24 hours, max 1000 attestations”) and displayed across multiple screens with a confirmation per screen. The user’s signing flow looks like reviewing a tax form rather than approving an opaque hash.

The encoding-design implication: the grant’s scope predicate must be expressible as a small, fixed-shape structure (schema set as a bitmap or sorted list of schema IDs; action set as a small enum). Free-form predicate expressions (the ERC-4337 EVM-bytecode approach) cannot be displayed safely; the user has no way to verify what they are approving. This is one of the reasons §3.3 fixes the scope predicate as a (schema set, action set) pair. Display-budget constraints fall out of the encoding choice; the encoding choice falls out of the protocol’s commitment to a verifiable scope semantics.

Future work here is integration work, not protocol work. The Mneme firmware roadmap covers grant-aware display modes; the v0.2 paper does not commit to a specific firmware UX, only to making the encoding amenable to one.

## 10.4 Quantum-Resistant Signatures

The §3 system model assumes classical signature schemes (Ed25519 or secp256k1). When the chain’s signature scheme upgrades to a post-quantum variant (Dilithium, Falcon, SPHINCS+), the delegation mechanism does not change semantically; the master and hot keys are both PQ keys, signatures verify under the PQ verifier, and the §5.5 inheritance proof is independent of signature scheme.

The operational complication is hybrid periods. During a chain-wide signature-scheme migration, the master key and the hot key may use different schemes (the master is upgraded first because masters are cold; the hot is upgraded later because hots are operationally constrained by client software). The grant object’s signature field needs to carry a scheme tag so the §4.3 admission check can dispatch to the right verifier. v0.2’s grant encoding allocates 8 bits for a scheme tag; the tag is currently fixed to “Ed25519” but can be extended.

Quantum resistance is also a concern for the grant’s nonce-based replay defense (§8.4): if the nonce scheme uses a collision-resistant hash, the hash must be PQ-collision-resistant. SHA-256 and SHA-3 both qualify under current PQ analyses. No change needed.

This subsection’s only forward-looking commitment: the grant encoding will reserve enough scheme-tag bits to enumerate the PQ schemes the chain might adopt over the next decade. The mechanism itself is signature-scheme-agnostic; no protocol change is needed when the chain upgrades.

## 10.5 Privacy-Preserving Delegation

The grant index is public chain state. Anyone can read it. This means the delegation graph (which masters delegate to which hot keys) is fully observable. For users running multiple agents (one Themisra agent for AI provenance, one Kleidon agent for SaaS, one Iris-relayed agent for autonomous flows), the graph reveals the user’s tool stack to anyone watching the chain.

The mitigation paths:

1. **Plausible deniability via shared operator addresses.** If an Iris-style relayer hosts thousands of users’ hot keys, a third party observing a grant from master  $M$  to hot key  $H$

where  $H$  is the well-known Iris address learns only that  $M$  uses Iris, not which Iris service  $M$  uses. This is the lowest-cost privacy mitigation: aggregate at the hot-key layer.

2. **ZK proof of grant existence.** A user issues a grant whose body is committed under a hash, and a zero-knowledge proof attests “I, the master, have issued a grant to hot key  $H$  with these properties, but the grant body is hidden.” The chain stores the commitment and the proof; the §4.3 admission check verifies the proof rather than reading the plaintext grant. This is the strong privacy variant; it requires a ZK proof system at admission-time cost, which contradicts the §6.3 light-overhead promise. Research-grade; not a v1 priority.
3. **Anonymous credentials.** The master key signs the grant blind, the hot key holds an anonymous credential, and the §4.3 check verifies the credential without learning the issuer. This is academically clean but requires a coordinator for credential issuance and a non-trivial cryptographic stack; we do not see a path to production-grade implementation in v1.

The v0.2 position: delegation graphs are public by default; users who need privacy compose at the application layer (use a single Iris hot key for all attestations, accept correlation via on-chain activity patterns). A privacy-preserving variant is on the research roadmap (paired with the cross-chain extension in §10.2, since both touch the grant encoding); not a v0.2 commitment.

---

## 11. Conclusion

Native delegation as a runtime primitive is the smallest mechanism that supports agent-on-behalf-of-user UX while integrating cleanly with PoUA reputation slashing. The smallest mechanism, not the most expressive: §3.3 deliberately constrains the scope predicate to a (schema set, action set) pair rather than the arbitrary EVM-bytecode predicate ERC-4337 affords. The constraint is what makes the §5.5 theorem provable, the §6.3 cost overhead bounded at  $\sim 10\%$ , the §6.4 light-client verification a single state-tree lookup, and the §8 security analysis tractable. Expressiveness costs would have multiplied across each of those surfaces; the constrained surface is the right surface for the agent UX this paper targets.

The paper’s three contributions resolve the three open questions the introduction posed. (1) **Mechanism:** `MsgDelegate` + `MsgRevokeDelegate` + grant index + admission check + lifecycle state machine, all specified in §4 with reference-quality detail. The mechanism is signature-scheme-agnostic, encoding-stable, and amenable to hardware-wallet integration under realistic display budgets. (2) **Slashing-inheritance theorem (§5.5):** under EV-maximizing adversaries with master-side risk-aversion  $\gamma > 1$ , the both-slashed rule with weights  $(w_m, w_h)$  satisfying  $w_m + w_h \leq 1$  and  $0 < w_h < w_m$  is the unique calibration that simultaneously satisfies P1 (master accepts delegation), P2 (master has monitoring incentive), P3 (hot operator faces cost), and P4 (no double-punishment). The recommended v0 calibration is  $(w_m, w_h) = (0.7, 0.3)$ . (3) **Empirical validation:** 88,200 Monte Carlo simulations confirm the theorem’s satisfying region across the full grid, and the P10 tail at the recommended calibration is well above the  $\mathbb{E}[U_{\text{master}}] \geq 0$  threshold (0.87, vs the P1 floor of 0). Users with bad-luck compromise-probability draws still find delegation rational.

The mechanism is the foundation for **Iris** (§7), Ligate’s MCP relay for autonomous agents. Iris composes native delegation (signing authority) with the per-schema fee market’s fee-payer primitive (payment authority) to ship the user-signs-once, agent-acts-for-T-seconds UX as a single coherent product. The §7.5 worked example (a Themisra session under Iris) traces every chain transaction back to the §4 admission check and the §5.5 slashing accounting; the composition is clean enough

that Iris’s commercial viability reduces to subscription-pricing discipline rather than protocol-level innovation.

The mechanism is also the foundation for any future product whose UX is “the user signed once, the agent acts for the next  $T$  seconds.” Themisra session-mode (one grant per AI provenance session), Kleidon’s TokenForge minting-bot (one grant per mint campaign), Mneme’s grant-issuance UX (the canonical interface for issuing grants), and the §9 incentive analysis verifies that honest delegation is the Nash equilibrium across all four parties (validators, users, agents, sponsors). Native delegation is not a special case of agent UX; it is the general primitive from which agent UX is built.

**What ships in v1.** The protocol mechanism (§4), the slashing-inheritance calibration (§5.5 at (0.7, 0.3)), the simulator (M1 + M2 covered, M3-M5 on the roadmap), Iris MCP integration (§7), and Mneme grant-issuance UX. The mechanism is feature-complete for the agent UX targeted in §1.1; subsequent v1.x releases will tune the calibration based on observed compromise rates (§5.5’s  $p_c$  stochastic adversary becomes empirical once the chain has live grants) and extend the test surface to the §10 limitations.

**What we are watching.** The §10 limitations name five extensions (recursive delegation, cross-chain, hardware-wallet display, PQ signatures, privacy-preserving variants) that are out of scope for v0.2 but on the research roadmap. Each is separable from the core mechanism; each can ship as a v0.3 or v0.4 increment without breaking v0.2 grants. The architecture is designed to compose with future extensions, not to require them.

**Invitations.** The paper, the simulator, and the chain implementation are all open to external review. The paper, simulator, and ligate-chain implementation tracker (issue #386) live in the ligate-io GitHub organization. Cold-asks for §5.5 theorem review are open through the PoUA reviewer channel at [hello@ligate.io](mailto:hello@ligate.io). Pilots of Iris under live grants on `ligate-devnet-1` are open to design partners; contact via the same channel. Feedback on the §10 limitations and the §11 v1 commitments is especially welcome before the calibration is locked in for mainnet.

The §1.1 thesis was that the chain primitive for agent UX is “the user signs once, the agent can sign on the user’s behalf for the next  $T$  seconds, scoped to actions  $A$ , with consequences  $C$  if the agent misbehaves.” This paper specifies  $T$ ,  $A$ , and  $C$  with enough rigor that a chain implementer can build it, a security analyst can audit it, and an agent vendor can integrate against it. The mechanism is small, the proof is tight, and the empirics support both. Native delegation is the chain primitive agents have been waiting for.

---

## References

### Account-abstraction patterns.

- Buterin, V., Weiss, Y., Tirosh, D., Nacson, S., Forshtat, A., Lundkvist, K., Wilson, H. (2023). *EIP-4337: Account Abstraction Using Alt Mempool*. Ethereum Improvement Proposal. <https://eips.ethereum.org/EIPS/eip-4337>
- Safe (Gnosis) (2017+). *Safe Smart Contract Wallet*. Documentation and contract source. <https://docs.safe.global/>, <https://github.com/safe-global/safe-contracts>
- Cosmos SDK Authors (2021). *Cosmos SDK x/authz module*. Cosmos SDK v0.43+. <https://docs.cosmos.network/main/build/modules/authz>

- Solana Labs (2020+). *Solana transaction format and fee-payer semantics*. Solana documentation. <https://solana.com/docs/core/transactions>

#### Delegation of economic security.

- Drake, J., Buterin, V., Edgington, B., Feist, D., et al. (2023). *EigenLayer: The Restaking Collective*. EigenLayer whitepaper. <https://docs.eigenlayer.xyz/eigenlayer/overview/whitepaper>

#### Companion Ligate Labs research.

- Ligate Labs (2026). *Proof of Useful Attestation: Consensus-Weighting Primitive for Attestation-Native Chains*. Working paper v0.8. <https://github.com/ligate-io/ligate-research/tree/main/papers/poua>
- Ligate Labs (2026). *Per-Schema Fees: Adaptive Fee Markets for Attestation Workloads*. Working paper v0.1. <https://github.com/ligate-io/ligate-research/tree/main/papers/per-schema-fees>
- Ligate Labs (2026). *Schema-Bound Tokens: AttestorSet as Mint Authority*. Working paper v0.1. <https://github.com/ligate-io/ligate-research/tree/main/papers/schema-bound-tokens>

#### Chain stack.

- Sovereign Labs (2024). *Sovereign SDK: A modular framework for sovereign rollups*. Documentation and source. <https://github.com/Sovereign-Labs/sovereign-sdk>
- Celestia Labs (2023). *Celestia: Modular Data Availability*. <https://celestia.org/learn/>
- Inter-Blockchain Communication (IBC) protocol specification. <https://github.com/cosmos/ibc>

#### Hardware wallets cited in §10.3.

- Ledger SAS (2014+). *Ledger Nano S/X/Stax: Hardware wallet specifications*. <https://www.ledger.com/>
- SatoshiLabs (2014+). *Trezor Model T / Safe 3: Hardware wallet specifications*. <https://trezor.io/>

#### Post-quantum signature schemes cited in §10.4.

- NIST (2024). *FIPS 204: Module-Lattice-Based Digital Signature Standard (Dilithium)*. <https://csrc.nist.gov/pubs/fips/204/final>
- NIST (2024). *FIPS 205: Stateless Hash-Based Digital Signature Standard (SPHINCS+)*. <https://csrc.nist.gov/pubs/fips/205/final>

#### Implementation references.

- Ligate Chain implementation. <https://github.com/ligate-io/ligate-chain> (native-delegation milestone tracked in issue #386).
- Native-delegation simulator. <https://github.com/ligate-io/ligate-research/tree/main/proto-types/native-delegation-sim> (M1 + M2 shipped; M3-M5 on roadmap).

## Appendix A: Simulator Reference

The reference simulator for the §5.5 theorem and the §4.4 lifecycle lives in the `ligate-research` repository under `prototypes/native-delegation-sim`. It is a Python 3.12 package. Core modules (under `src/native_delegation_sim`):

- `validator.py`: closed-form expected-utility functions for the master and the hot key under the both-slashed rule.
- `grant.py`: `Grant`, `InheritanceRule`, and the four predicate evaluators P1-P4.
- `slashing.py`: `apply_slash()` and the `SlashOutcome` enumeration.
- `lifecycle.py`: the §4.4 state machine (`GrantLifecycle`, `GrantState`).
- `strategy_search.py`: Monte Carlo grid sweep over  $(w_m, w_h)$  under a `StochasticAdversary` with  $p_c \sim \mathcal{N}(\mu, \sigma^2)$  clipped to  $[0, 1]$ .

Additional surface:

- `scripts/run_theorem_1_validation.py`: produces the  $21 \times 21$  grid  $\times$  200 seeds = 88,200 simulation heatmap cited in §5.5. Output: `out/theorem_1_validation.png`.
- `tests/`: 56 unit tests covering predicate evaluators, lifecycle transitions, strategy-search determinism, and percentile invariants.

**Running the simulator.** From the package root:

```
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
pytest                                     # 56 tests, all pass
python scripts/run_theorem_1_validation.py # writes out/theorem_1_validation.png
```

**Coverage matrix.**

| Paper section    | Sim. module                              | Tests                                     |
|------------------|------------------------------------------|-------------------------------------------|
| §4.4 lifecycle   | <code>lifecycle.py</code>                | <code>test_lifecycle.py</code>            |
| §5.5 P1-P4       | <code>grant.py</code>                    | <code>test_slashing_inheritance.py</code> |
| §5.5 region      | <code>strategy_search.py</code>          | <code>test_strategy_search.py</code>      |
| §5.5 calibration | <code>run_theorem_1_validation.py</code> | smoke check                               |

**What is not yet in the simulator (M3-M5 roadmap).** Cross-language test vectors for the canonical grant encoding (M3), strategic-adversary extension where the hot key chooses misbehavior actions to maximize  $G_{\text{misbehave}} - w_h \cdot \Lambda$  (M4), and full chain integration test against a `ligate-chain` testnet (M5). Each milestone is tracked in the simulator’s `ROADMAP.md`; pull requests welcome.

## Appendix B: Formal Definitions

We collect the formal definitions used throughout the paper in one place for reference.

**Definition (Master key).** A keypair  $K_m = (\text{sk}_m, \text{pk}_m)$  whose public key  $\text{pk}_m$  uniquely identifies the user on chain. PoUA reputation accrues to the address derived from  $\text{pk}_m$ . The signing key

$\text{sk}_m$  is held cold (offline, in hardware, or under a multisig); the user signs grants and high-value transactions directly with  $\text{sk}_m$ .

**Definition (Hot key).** A keypair  $K_h = (\text{sk}_h, \text{pk}_h)$  held online by an agent (or operator running multiple agents). The hot key has no inherent on-chain identity; its on-chain authority derives entirely from grants issued by master keys.

**Definition (Grant).** A tuple  $G = (M, H, \Sigma_G, A_G, T_{\text{start}}, T_{\text{end}}, R_G, \text{nonce}_G, \sigma_M)$  where:

- $M$  is the master address (derived from  $\text{pk}_m$ ).
- $H$  is the hot address (derived from  $\text{pk}_h$ ).
- $\Sigma_G \subseteq \Sigma_{\text{chain}}$  is the scope predicate's schema set (a subset of registered schemas).
- $A_G \subseteq A_{\text{chain}}$  is the scope predicate's action set (a subset of valid chain action types: `SubmitAttestation`, `RegisterSchema`, etc.).
- $T_{\text{start}}, T_{\text{end}} \in \mathbb{N}$  are chain heights bounding the grant's validity window.
- $R_G$  is the inheritance rule (an element of  $\{\text{master-only}, \text{hot-only}, \text{both-slashed}\}$ , with `both-slashed` being the §5.5 recommended default).
- $\text{nonce}_G \in \mathbb{N}$  is a per-master strictly-increasing nonce.
- $\sigma_M$  is the master's signature, under  $\text{sk}_m$ , over the canonical encoding of the tuple  $(M, H, \Sigma_G, A_G, T_{\text{start}}, T_{\text{end}}, R_G, \text{nonce}_G)$ .

**Definition (Scope predicate).** The pair  $(\Sigma_G, A_G)$ . A transaction  $\tau$  targeting schema  $s$  with action  $a$  is in scope if and only if  $s \in \Sigma_G$  and  $a \in A_G$ .

**Definition (Time-bound).** The pair  $(T_{\text{start}}, T_{\text{end}})$ . A transaction  $\tau$  included at height  $h$  is in-window if and only if  $T_{\text{start}} \leq h \leq T_{\text{end}}$ .

**Definition (Inheritance rule).** A function  $R : \text{SlashEvent} \rightarrow (\Delta_m, \Delta_h)$  that maps a slashing event of base magnitude  $\Lambda$  to a pair of reductions  $(\Delta_m, \Delta_h)$  to be applied to the master's and hot's reputations respectively. The §5.5 recommended rule is *both-slashed* with  $\Delta_m = w_m \cdot \Lambda$  and  $\Delta_h = w_h \cdot \Lambda$  where  $(w_m, w_h) = (0.7, 0.3)$ .

**Definition (Compromise probability).** A random variable  $p_c \in [0, 1]$  representing the probability that the hot key is compromised during the grant window. The deterministic §5.5 analysis treats  $p_c$  as a fixed parameter; the simulator's M2 stochastic-adversary extension models  $p_c \sim \mathcal{N}(\mu, \sigma^2)$  clipped to  $[0, 1]$ .

**Definition (Master risk-aversion).** A constant  $\gamma > 1$  representing the master's risk-aversion coefficient over reputation loss. The §5.5 theorem requires  $\gamma > 1$  so that P1's  $\mathbb{E}[U_{\text{master}}] = G_{\text{delegate}} - \gamma \cdot p_c \cdot w_m \cdot \Lambda \geq 0$  tightens the constraint from a risk-neutral  $\gamma = 1$  baseline.

**Definition (Four properties P1-P4).**

- **P1 (Master accepts delegation):**  $\mathbb{E}[U_{\text{master}}] \geq 0$ .
- **P2 (Master has monitoring incentive):**  $w_m > 0$ .
- **P3 (Hot operator bears cost):**  $w_h > 0$ .
- **P4 (No double-punishment):**  $w_m + w_h \leq 1$ .

The §5.5 theorem proves: the both-slashed rule with  $(w_m, w_h)$  satisfying  $w_m + w_h \leq 1$  and  $0 < w_h < w_m$  uniquely satisfies P1-P4 simultaneously under EV-maximizing adversaries with  $\gamma > 1$ .